RIGA TECHNICAL UNIVERSITY

Faculty of Computer Science, Information Technology and Energy

Institute of Applied Computer Systems

Pankaj Chettri

Student of academic master level study program "Business Informatics"

Student ID No 221AEM001

METHOD FOR DETECTING ROOT CAUSES OF DEFECTS DISCOVERED DURING SOFTWARE TESTING

MASTER THESIS

Scientific adviser

Dr.sc.ing., assoc. professor

Ē. Nazaruka

RIGA 2025

Kvalificēts elektroniskais paraksts

PANKAJ CHETTRI

2025-01-07 00:09:34 GMT+2

Mērķis: Paraksts

Powered by Dokobit

RIGA TECHNICAL UNIVERSITY

FACULTY OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY

Institute of Applied Computer Systems

Business Informatics

Work Performance and Assessment Sheet of the Master Thesis

The author of the graduation thesis:

Student Pankaj Chettri _____

(signature, date)

The graduation thesis has been approved for the defence:

Scientific adviser:

Dr.sc.ing, Associate Professor, Ērika Nazaruka _____

(signature, date)

ABSTRACT

ROOT CAUSE ANALYSIS, DEFECT PREDICTION, MACHINE LEARNING IN SOFTWARE TESTING, METADATA FEATURE EXTRACTION

With the growing popularity and demand for enhanced software functionality, the process of creating and maintaining the software functionalities is regarded to be crucial. There have been significant paradigm shifts in software testing and defect management to foster improved speed and accuracy, but identifying the root causes for triggering issues remains a challenge, limited by their static, manual nature that results in inefficient resource optimisation, higher expenses and delays.

This thesis investigates the area of Root Cause Analysis (RCA) in software testing, with an emphasis on developing a technique for detecting the root causes of issues during software testing. In solving the limitations, it proposes an efficient RCA approach by leveraging the Meta-Data Analysis and Machine Learning (ML) techniques implemented using Python. The main intent is to provide an efficient approach for finding the root causes of defects discovered during the software testing and providing insights in determining the preventive measures to decrease or eliminate them in the future, hence enhancing the software development quality.

Information on the volume of the thesis on current progress– 91 pages, 19 tables, 15 figures, and 77 sources of reference.

ANOTĀCIJA

SĀKŅCĒLOŅU ANALĪZE, DEFEKTU PROGNOZĒŠANA, MAŠĪNMĀCĪBA PROGRAMMATŪRAS TESTĒŠANĀ, METADATU ĪPAŠUMU IEGŪŠANA

Pieaugot popularitātei un pieprasījumam pēc uzlabotas programmatūras funkcionalitātes, programmatūras funkcionalitātes izveides un uzturēšanas process tiek uzskatīts par izšķirošu. Programmatūras testēšanā un defektu pārvaldībā ir notikušas ievērojamas paradigmas izmaiņas, lai veicinātu ātruma un precizitātes uzlabošanos, taču problēmu rašanās pamatcēloņu noteikšana joprojām ir izaicinājums, ko ierobežo to statiskais, manuālais raksturs, kas rada neefektīvu resursu optimizāciju, lielākus izdevumus un aizkavēšanos.

Šis darbs pēta pamatcēloņu analīzes (RCA) jomu programmatūras testēšanā, liekot uzsvaru uz tehnikas izstrādi problēmu pamatcēloņu noteikšanai programmatūras testēšanas laikā. Risinot ierobežojumus, tā piedāvā efektīvu RCA pieeju, izmantojot metadatu analīzes un masiņ mācīšanas (ML) metodes, kas ieviestas, izmantojot Python. Galvenais mērķis ir nodrošināt efektīvu pieeju programmatūras testēšanas laikā atklāto defektu pamatcēloņu atrašanai un ieskatiem preventīvo pasākumu noteikšanā to samazināšanai vai novēršanai nākotnē, tādējādi paaugstinot programmatūras izstrādes kvalitāti.

Informācija par darba apjomu par pašreizējo progresu – 91 lappuses, 19 tabulas, 15 attēli un 77 uzziņu avoti.

TABLE OF CONTENTS

INTRODUCTION 5
1. RESEARCH METHODS AND RCA METHODOLOGIES 7
1.1 Root cause analysis in software testing 9
1.1. Traditional RCA approaches and their inherent constraints 12
1.2.1 Elements of RCA 12
1.2.2 Limitation analysis of traditional RCA approaches 14
1.3 Transition to automated RCA methodology 18
2. METADATA EXTRACTION AND DATA PREPERATION 23
2.1 Metadata extraction 23
2.1.1 Selection of project management software 23
2.1.2 Data extraction 24
2.2 Data Preparation 27
2.2.1 Metadata standardization 27
2.2.2 Metadata cleaning 32
2.2.3 Analysis of system parts under development for vulnerability 34
3. PROPOSED ML-BASED RCA MODEL 41
3.1 Related work 41
3.2 Features and target selection 41
3.3. Model selection 44
3.3.1 Proposed learning model 44
3.3.2 Evaluation metrics 45
3.4. Model training 46
3.5. Analysis of model performance 47
3.5. Results and discussion 52
4. POTENTIAL APPLICATION OF PROPOSED MODEL 56
4.1 Incorporation of rule-based RCA layer 56
4.2 Linkage between rule-based and root causes 58
4.3 Expected outputs interpretation and recommendations 59
CONCLUSION 61
LIST OF REFERENCES 64
APPENDIXES
Appendix 1. Code used for data preprocessing and cleaning
Appendix 2. Code used without Hyperparameter tuning
Appendix 3. Code used with Hyperparameter tuning

INTRODUCTION

With the rapid implementation of digital transformation and the need for robust software systems, it introduces increasingly sophisticated architecture that creates additional difficulties to software organisations in the ability to manage and sustain the software quality and reliability.

Root Cause Analysis (RCA) is crucial to enhance the software dependability and to ensure the defects do not persist in the future releases by effective identification and validation of the root causes surfaced in the software along with an effective preventive actionable insight rather than just for the symptoms. Traditional RCA methodologies are resource-intensive and generally unreliable to deal with the vast amount and intricacies, especially for organisations with limited resource availability [1], [2], [3].

An effective RCA approach is required to address these limitations and for the proactive risk management and quality control in software testing. Leveraging the use of Metadata Analysis (MA) coupled with Machine Learning (ML) can shift RCA from a manual, retrospective process to an automated, predictive one and reducing the potential biases and time necessary for identification of the root causes.

As the advantages of ML-based RCA is certain, there are several challenges imposed for implementing that includes the multifaceted nature of ML training, data collection and preprocessing necessities, the need for organisational changes to integrate new RCA techniques in existing workflows and its scalability and customization flexibility [3], [4], [5], [7]. The author of this thesis aims to purpose and implement efficient automated RCA method for organisations that aids in identification of root causes and reduce recurrence of defects.

The novelty of this approach is based on coupling of the MA from real-world defect reports with a supervised learning-based RCA model to streamline the RCA process and benefit software testing. The central idea is that by training, testing and validating the ML-based RCA model using MA of the contextual defect data to substantially improve the defect detection and lower the probability of recurrence when compared to the traditional RCA methodologies.

The **main research question** of the thesis is the following:

To what extend does the ML-based RCA model addresses the limitations of traditional RCA approaches?

The **goal of this thesis** is to develop an effective method to detect the root causes of defects during software testing and offering insights that aids in applying preventive measures and to reduce or mitigate the recurrence aiding in improved software development quality. To accomplish this goal, the following tasks are established:

- Perform review and analysis of existing methods for RCA of software defects
- Extract metadata of real-world datasets and create the training, testing and validation datasets
- Develop the method for detecting root causes of defects based on the created datasets
- Evaluate the proposed method using the validation dataset
- Analyse the results and compare with the existing research results
- Provide final recommendation on improving the developed method.

The outlined structure of the thesis follows:

1) **Chapter 1** provides an in-depth literature review and analysis of RCA principles, current methodologies and the challenges involved with root cause identification in software testing that serves as a basis for better understanding of RCA methodologies and structural knowledge that will be considered in the experimental part of this research
2) **Chapter 2** discusses the extraction and preparation of the data how the training, testing and validation datasets was produced to build the RCA model
3) **Chapter 3** covers the method development including the design and implementation of the ML-based RCA model. This section describes the used algorithmic approach and defect-linking process used. It further examines the model's performance evaluation using key indicators and its effectiveness in software testing, including the comparison to the traditional RCA approaches.
4) **Chapter 4** includes the conclusion that covers the impact of the findings, the anticipated influence on software testing, research limitations and potential directions for further research directions.

Appendix 1 provides code used for data preprocessing, Appendix 2 and Appendix 3 provides code used for RCA-model development with and without hyperparameter tuning.

1. RESEARCH METHODS AND RCA METHODOLOGIES

This chapter discusses the fundamental research approach employed throughout the thesis, along with the research method used to perform the literature reviews. Furthermore, it considers the importance of determining and implementing an appropriate automated RCA approach, while also considering the influence it has on the identification of defects in comparison with conventional RCA methodologies.

It also describes the thesis's comprehensive research methodology, along with the research questions required to effectively frame the study towards its intended primary purpose. It likewise describes the research approach utilised to conduct a thorough literature review to develop and summarise knowledge regarding RCA and its use in defect identification and mitigation, which will be needed for the experimental part of this study. Section 1.1 and 1.2 investigates RCA taxonomies, exposing the limitations of traditional RCA methodologies and how it could potentially overcome by the selection of best-fit automated RCA technique. Section 1.3 provides a holistic view of the existing RCA methodologies and research gaps through literature review which is needed for our experimental part of this study.

To structure the overall study to be in-context with the central aim and primary research question, a comprehensive research methodology is needed.

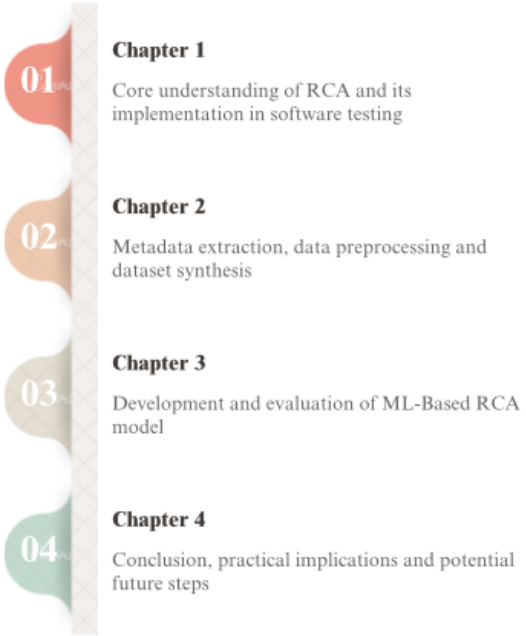

Fig. 1.1. Holistic thesis framework

Figure 1.1 shows the link between the chapters in this thesis. The literature review is a continual during the whole thesis. Chapter 1 provides the required theoretical knowledge, Chapter 2 deals with the data extraction and preprocessing, Chapter 3 and Chapter 4 discusses the method development and evaluation, practical implications and potential future steps.

Several research questions are imposed by the author to structure and propel the study towards achieving the primary goal:

- What are the limiting factors of traditional RCA methodologies and how it can be addressed using automation and ML?
- How can organisations implement an efficient and automated RCA method even with constraints such as limited or incomplete data?
- How can the defect metadata be utilized systematically to train ML-based RCA model?

To provide in-context literature review and analysis, the following strategy as shown in figure 1.2.

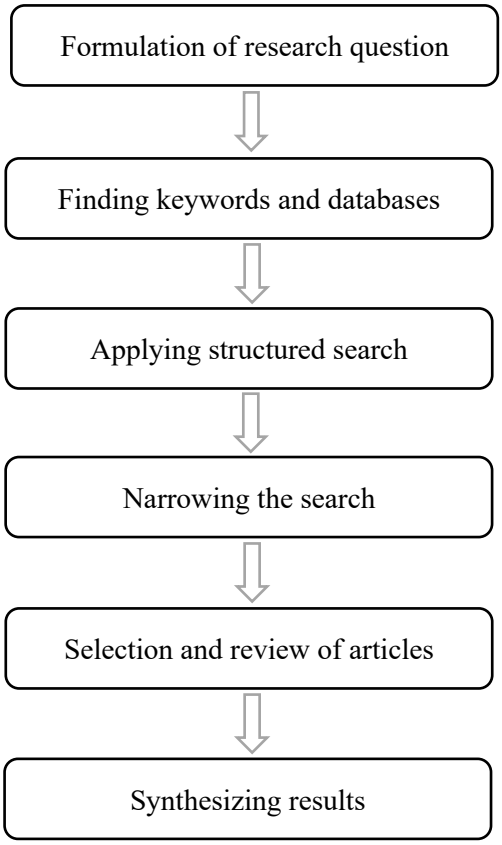

Fig. 1.2. Research method for literature review

After the formulation of research questions, a review of the topic was carried out in various online databases namely IEEE, InfoQ, ScienceDirect alongside referencing online web materials. The search was narrowed down to only journals, articles and research papers after 2017. A total of 35,481 articles were identified and the keywords used were *defect prediction in software testing, root cause analysis techniques, root cause analysis, machine learning in software testing, metadata feature extraction*.

Total of 90 articles were selected based on the relevance with the title and further screening was conducted based on their relevance to research questions were done. A total of 61 articles were selected for the in-depth literature review and analysis to develop the required knowledge for the gap identification and the best-fit automated RCA approach which will be needed for our experimental part of this study.

1.1 Root cause analysis in software testing

Software product reliability and stability drive growth while also promoting efficient resource optimization and greater customer satisfaction. However, in the constantly evolving environment of business operations, quality management becomes increasingly challenging resulting in deliverables that are often vulnerable to operational inefficiencies or faults. It is critical for every organization, regardless of domain, to effectively identify and solve the underlying defect causes to sustain growth, increase productivity, and to improve market competence.

Delivering dependable software has become a significant challenge for successful software-development companies and even with advancements in software testing processes for better testing coverage, it continues to be apparent that certain areas of the software system are more vulnerable to vulnerabilities than others [8].

Proper defect identification and mitigation can have a significant impact on software development processes and in ensuring higher quality deliverables. However, due to the resource-constrained nature of software development, it is often impossible to catch all the defects during software staging phase and their recurrence in the future. It is seen on the study [9], that the software testing consumes more than 50% of the overall development cost and the effort required is approximately 40-60% of the overall development process. It can be understood that there is an imbalance in the utilization of resources which is highly dedicated for defect detection.

As a result, root cause analysis (RCA) is a technique that not only aids in the discovery of underlying causes but also goes further resolving the immediate indicators of a defect to identify the fundamental causes which triggered the failure. RCA helps to classify the contributing factors and behaviours to a scenario. All the detected causes are grouped into categories so that the defect or class of defects can be separated, and proactive strategies may be developed in an effective way while preventing their reoccurrence.

A simple definition of RCA is "*is the process of inferring the set of faults that generated a given set of symptoms*" [10]. Proactively identifying and resolving faults along with root cause analysis, leads to software systems that are reliable, secure, and meet user expectations. It is proving to be vital for maintaining the software quality and fostering excellence in the constantly evolving software development operations.

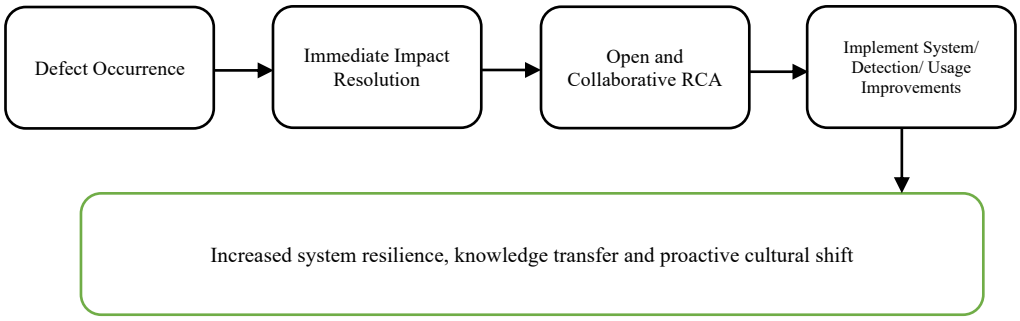

Fig. 1.3. Learning from unintended failures pattern (adopted from [11]**)**

The main purpose of RCA is to identify and solve the underlying causes of issues, hence reducing their reoccurrence. For example, in the aviation sector, rigorous RCA during simulation testing of Boeing 777 complete flight simulators has resulted in increased safety and dependability by revealing and correcting hidden faults [12].

Premortem analysis, a preventive approach to RCA, aids in predicting potential issues before they occur. PayPal's engineering teams used a customized premortem analysis to predict project failures and to identify potential causes. This method enabled them to identify problems early on, promoting a culture of psychological safety and avoiding costly mistakes down the road [13].

RCA could potentially be applied to cost management. For example, AWS used cost anomaly detection to identify and reduce unexpected financial spikes, which improved budget adherence and operational efficiency[14]. Learning from unexpected failures may also improve process optimization and resource utilization, as shown in a

variety of technological applications [11]. Furthermore, RCA's use in numerous sectors proves its adaptability. Examples from industry and healthcare demonstrate how RCA may tackle urgent issues while also contributing to long-term improvements, assuring sustainability and dependability [15].

It is evident with the use cases of RCA that the organizations can address systemic issues leading to fewer defects in future iterations, improving the deliverable quality and reliability by identification and resolving quality bottlenecks, proper resource utilization and ultimately meet customer expectations and foster trust and satisfaction.

It is particularly valuable in Agile and DevOps environment where teams operate under the tight deadlines, rapid release cycles and the cost of fixing defects post-deployments can be high which could lead to poor resource optimization and decreased customer satisfaction.

RCA is widely acknowledged as one of the best techniques in quality management considering that it moves the emphasis from simply fixing issues to avoiding them, which is in complete agreement with the values of continuous improvement and proactive quality assurance, reaching the peak in late 2022, early 2023, according to Google Trends results (Fig. 1.4).

Fig. 1.4. Root cause analysis searches on the web. Google Trends, October 2014 – October 2024.

With this understanding, the IT development teams can implement informed decision making to address the defects and prevent subsequent catastrophes, while fostering continuous learning in enabling the teams to make informed decisions and avoid repeating mistakes.

1.1. Traditional RCA approaches and their inherent constraints

With the understanding about the applications and outcome of RCA, it is distinct about the merits and that the RCA serves as the foundational quality management in software testing.

1.2.1 Elements of RCA

It follows a structured sequence of procedures employed to identify, analyse, and address the root causes of defects and each of these steps provide a fundamental structure that can be applied across multiple RCA methodologies, allowing a comprehensive approach in identifying and mitigation of the discovered defects [16].

The following are the five fundamental elements of RCA:

- **Anomaly identification and definition**

The root cause commences with the identification of the underlying issue which is set to provide a structured analysis path. It is carried out by capturing and documenting the defect along with the triggers and its effect on both functional and performance aspects. Proper definition of the scope enables team to funnel the issue towards the main cause, rather than utilizing the resources for causal causes.

This stage comprises of the data gathering for effective understanding of the defect's scope and its relevance.

- **Data Collection:**

After the definition of the issue, the next vital step is to gather the needed data to have a clear visual of the defect. It involves in collecting the historical data such as defect logs or test reports for gaining valuable insights of the concerned defects and the contributing variables. It comprises of the artefacts that gives data-rich information regarding the context. This is a crucial step enabling the effective evaluation of the defects and the follow-up identification and verification of the possible root causes.

- **Cause Identification:**

The brainstorming for the possible reasons or causes of the defects, it is then categorised into different categories such as direct reasons of the defect, the contributary reasons of the defect or the root cause of the defect. It comprises of in-

depth brainstorming with the utilization of various RCA tools available such as FMEA, Fishbone, 5 Whys etc for understanding and funnelling towards the root cause of the defect. Further, analysis of the triggers is done within the development team in themes such as Code, UI/UX, Checkout, Insufficient understanding etc.

- **Root Cause Verification and Analysis:**

After the possible causes have been identified, the final step is about understanding the potential root cause of the defect. This step comprises of rigorous analysis and investigation to whether the causes' hypothesis actively contributes to the defect. Validation from the analysis part can further be carried out in a contained environment for ruling out irrelevant causes.

This phase ensures the corrective and/or preventive actionable insights are based on the root causes and not on surface level symptoms rather while mitigating the data sparsity and personnel biases. This is the most vital part to determine and validate that the actionable insights are effective.

- **Solution Development and Implementation:**

The RCA aims in the creation of effective and actionable solutions to effectively address the underlying cause(s) of the emerged defect. Collaboration and cross communication across teams regarding the corrective and preventive measures are done keeping in mind the resource constraints and the technological infrastructure and expertise.

The agreed solution is then implemented. Further, the validation of the fix post deployment is vital that may comprise of targeted testing or feedback. This final stage of the RCA ensures and mitigates the defect reoccurrence. Documentation is vital to capture the preventive measures taken for the continuous improvement such as introducing new quality metrics, improving knowledge in both coding and testing practices.

This stage of the RCA aims in applying the corrective actions to the defect while also facilitating the preventive measures in mitigation of such defects to emerge in future instances.

1.2.2 Limitation analysis of traditional RCA approaches

In this subsection, the author aims in analysis of the existing RCA tools after the discussion of the general RCA approach. It is of utmost importance that the selection of significant tools as per the business requirements can greatly optimise the internal processes and producing the high-quality products and meeting the user expectations.

As the technological advancement, several RCA approaches have emerged with each having strengths, purpose and altering challenges [17], [18], [19].

1. 5 Whys:

The 5 Whys technique is a progressive way to identify the root causes of problems, particularly in business environments. It involves asking "Why?" until the root cause is identified, assisting businesses perceive systemic difficulties and deliver focused solutions. This technique focuses on the deep analysis and identification of root causes along with pinpointing structural limitations [20], [21].

***Analysis**:* The study is shallow since it is not adequate to properly identify the complex root causes which could be overlooked due to human insights and biases. The multifaceted causes may be failed to recognize as it follows a linear path, ignoring boundary systematic issues.

2. Fishbone/Ishikawa diagram:

The Ishikawa Diagram, commonly referred to as the “Fishbone Diagram” is an illustration used for identifying and categorizing the root causes of defects. It is effective in dealing with diverse issues such as production delays, client complaints and operational ineffectiveness, as well as in prioritizing activities [22], [23], [24].

***Analysis**:* It is confined to direct causes and lacks quantitative evaluation that makes it time-consuming and perhaps triggering a loss of value when not managed successfully. It also might be unable in understanding the root causes of interrelated systems.

3. Fault Tree Analysis (FTA):

It top-bottom approach for determining the root causes of failure in complex architectures. It entails in producing a visual representation of the rational relationships between defect and their causes that lead to an identifiable final analysis. This aids in identification of the crucial feature combinations that contribute to the defect. Teams engage to discover potential causes that contribute and their associations resulting in-depth understanding of vulnerabilities. It also sets businesses to set priorities for a proactive approach in defect mitigation measures by emphasizing on the most relevant triggers [21], [25].

***Analysis**:* Time consuming, data-intensive and challenging to cope with as functionality and processes evolve which makes it challenging to uphold in agile software development framework.

4. PROACT® RCA Method:

It is a structured strategy for solving defects that includes gathering of relevant information, conducting an in-depth investigation with an array of tools and determining root causes. It focusses on immediate identification of underlying root causes and in developing proactive measures to mitigate the defect recurrence. It requires cooperation and cross-functional participation while taking into consideration of varied perspectives and encourages businesses to take a proactive approach in addressing system or process defects [18], [21], [26].

***Analysis**:* Time-consuming, data-dependent and necessitates specialized training with detailed data which may not be readily available, especially for businesses with limited data availability.

5. Failure Mode Effect Analysis (FMEA):

FMEA is an approach in identifying defect possibilities in a system, product or processes with emphasis on Risk Priority Number (RPN) which is a multiple of the severity, frequency and detectability. This allows development teams to assign priority to defects by considering their potential effect and enabling them to recognize risks, lay emphasis on mitigation actions in improving overall system or product resilience [21], [27], [28].

It is particularly beneficial for implementation of novel software or services and to address the significant changes to existing module's architecture and scalability. It

might be performed during the early phases of product development or process design, allowing business to proactively manage the defect risks and improve reliability and performance [21].

***Analysis**:* Subjective to bias during quantitative scoring resulting in inconsistent and poor prioritization, poor knowledge about emerging defect patterns lead to bypassing of new defects.

6. Pareto Analysis:

It is based on the Pareto Principle which is a strategy in identifying and prioritizing the most significant factors contributing to a defect. It centres on the "vital few" defects which enables businesses to prioritize resources effectively by collecting metrics, classifying triggers through the utilization of a Pareto chart [5], [29], [30].

***Analysis**:* For atypical systems where the defects are evenly spread, it may result in presumption asymmetry as it follows that 80% of defects originate from 20% of triggers which is a proximity and could miss nuanced root cause distribution that shows significant variance. Moreover, it has the possibility to identify concentrated defects and fails to reveal on how the causes or the interconnection of related defects.

7. Current Reality Tree (CRT):

It is one of the RCA tools created by Dr. Eliyahu Goldratt as part of the "*Theory of Constrains*" methodology. It is intended to identify the defects and their root causes while emphasizing distinctive contributory routes of defects [31], [32], [33].

***Analysis**:* Complex and occasionally cumbersome with numerous multifaceted defects which makes it difficult to interpret and sustain for large dynamic system and processes. The mapping of defects causes maybe subjective and misinterpreted as it is based on the analyst's judgement and expertise which is challenging as it demands an in-depth knowledge of the entire system architecture.

8. Affinity Diagram:

Affinity Diagram is a tool for problem solving and collaborative analysis that organizes and synthesizes immense quantities of information into natural links. It enables teams to obtain an extensive understanding of complex issues and find the root causes clustered by themes such as UI glitches, backend faults and database connectivity pertaining defects. It is particularly valuable when dealing with multifaceted difficulties involving several stakeholders or complex data layers and it

promotes collaboration, identify areas of priority, and achieve consensus on vital operations by visually mapping related ideas and concepts [21], [34].

***Analysis**:* Manual effort is required for sorting and classification which to prone to be subjective and oversights as it is dependent on the business expertise and boarder causes may be overglanced if seen unfitting into already existing categories. Moreover, the used diagram is static and not evolving with system, it requires frequent maintenance to be in-context with the emerging defects.

9. Scatter Diagram:

A scatter diagram (or scatter plot) depicts the relationship between two variables. It aids in detecting patterns, connections, and possible cause-and-effect linkages [31], [32], [33].

***Analysis**:* Intuitive understanding of changes effect on other factors and focus on correlation and not causation.

10. DMAIC:

It provides a roadmap for businesses to enhance existing processes or solve reoccurring defects. It comprises of 5 stages define, measure, analyse, improve and control. The first step entails in defect definition, data collection and analysis, solution design and implementation and maintaining progress through effective control mechanisms [35], [36].

***Analysis**:* Time-consuming, inflexible, puts forth cultural resistance and have high data dependency which is especially a challenge for businesses with limited data availability.

Overall, each approach is seen to have significant benefits but with limitations such as bias, monotonous data representation, resource/time constraints and sophistication makes it difficult in real-world applications especially in-context of the businesses with limited data availability.

This all paves the absolute necessity for an automated, data-driven RCA approach which can capture complex and evolving defect relationships while also ensuring the accuracy and efficiency in RCA by reducing manual effort and human bias in sophisticated and resource constrained development environment such as agile and DevOps resulting in effective resource utilization and fostering productivity.

1.3 Transition to automated RCA methodology

After getting familiar with the necessity of root cause analysis in software testing and the overall software development, it can be understood that among the quality attributes such as usability, agility and maintainability etc., reliability is seen to be the most significant as its absence runs systems and processes out of sequence, resulting in poor customer retention and catastrophic resource utilization. It was also understood that the ability to identify the defect and its mitigation from recurrence is vital and which is achievable with RCA.

However, with the limitation of traditional RCA methods, an automated RCA can alleviate the limitations of human bias, stagnant adaptation, scalability and lagging accuracy of defect detection and its prevention from recurrence with leveraging the machine learning (ML) capabilities resulting a paradigm shift towards continuous improvement and fostering proactivity and supporting both defect identification and its prevention in a dynamic environment.

Advancement in software systems and their processes provide complex architectures necessitating enhanced RCA for informed human data-driven decision making. Software testing seeks to detect defects whereas software defect prediction (SDP) enhances software quality and effective utilization of available resources and defined as "*using models that indicate the areas of the code where faults are most probable*" [37] which further adds "*many ML techniques have been conceived, each with different effects based on the circumstances and no universal model could be applied to all datasets to develop effective solutions.*"

SDP is vital as in predicting the quantity of defects in software components can help classify them and provide more testing resources for modules that may have additional shortcomings [38]. This allows in more efficient testing resource allocation. With the high-speed quantity of log data produced, it is difficult to comprehend due to the unstructured formatting, unintuitive for human readers, automated or model-based RCA outperforms manual RCA in the aspect of speed and cost [39].

It is evident that traditional metrics and processes are inadequate for managing complex systems which makes it a necessity in automation for precise root-cause hypothesis creation. Moreover, despite substantial study, there is still inadequate knowledge about the effective algorithm adaptation for advanced dynamic

development environments which requires scalability and real-time responsiveness [10], [37], [42], [43].

In the array of ML techniques, understanding and selection of an effective ML algorithm can deeply benefit the businesses resulting in effective RCA and SDP while it could also have inverse effect of poor resource optimization, add complexity and catastrophic internal development process, if not address properly.

Several classification schemes for the base classifier were examined by [10] that aids in the understanding of top classifiers for SDP through the analysis of 132 papers and examining the classifications categories as demonstrated in figure 1.4.

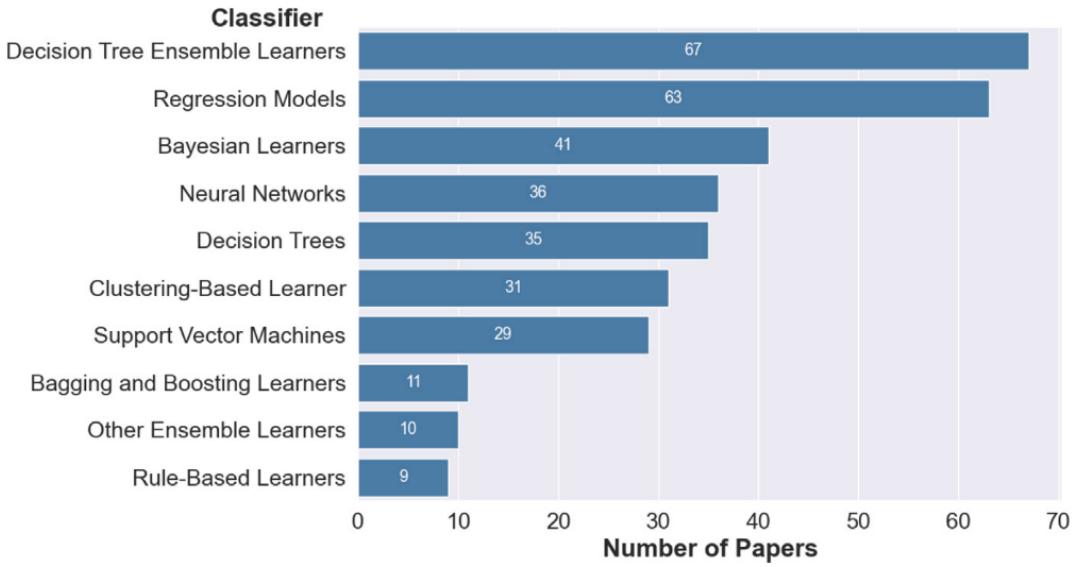

Fig. 1.5. Top classifiers for SDP (adopted from [10])

It was seen that the "decision tree ensemble learners" such as *Random Forest, Hoeffding Tree Ensemble, Gradient Boosting*, was the most widely used classifier followed by "Regression models" and "Bayesian Learners"

The reasons for the selection of the classifiers were diverse but [8] suggests that the selection was chosen based on performance in benchmarking studies and the pertaining knowledge transfer in the SDP research community.

It was found out in the study [37] that the top 5 techniques in ML SDP in terms of popularity were *decision trees, Nearest Neighbour Search (KNN)*, *Support Vector Machines (SVM), Random Forest (RF) and Neural Networks.*

The research paper [44] shows the Effort-Aware Defect Prediction (EADP) model in accordance with the predicted defect density with the use of Line of Code (LOC) as an alternate for effort. EADP was defined as a task of ranking of software modules based on defect density and number of defects.

The core component of the models was the learning to rank (L2R) algorithm. Table 1 represents the overview of parameter values of L2R algorithms, adopted from [44].

Table 1. Parameter value of L2R algorithms (adopted from [44])

Parameter value overview of the L2R algorithms.

Family		Label	Algorithm[a]	Parameter description[b]	Candidate parameter values (Default value is in bold)
Classification based Pointwise Approach	Bayes	C1	Naïve Bayes (NB)	[N] Laplace Correction (0 indicates no correction).	**{0}**
	Function	C2	Logistic Regression (LogR)	[N] Tolerance for stopping criteria.	{0.1,0.01,0.001,**0.0001**,0.00001}
		C3	Simple Logistic (SL)	[N] The number of iterations for LogitBoost.	**{500}**
		C4	Radial Basis Functions Network (RBFNet)	[N] Ridge factor for quadratic penalty on output weights.	{0.1,**0.01**,0.001, 0.0001, 0.00001}
		C5	Sequential Minimal Optimization (SMO)	[N] Penalty parameter of the error term with linear kernel.	{**1**}
	Tree	C6	Classification and Regression Tree (CART)	[N] The minimum number of samples required to split an internal node.	{**2**,3,4,5,6}
		C7	C4.5	[N] The confidence factor that are used for pruning (smaller values incur more pruning).	{**0.25**}
		C8	Logistic Model Tree (LMT)	[N] The number of iterations for LogitBoost.	{**500**}
		C9	Random Forest (RF)	[N] The number of trees in the forest.	{**10**,20,30,40,50}
	Lazy	C10	K-Nearest Neighbors (KNN)	[N] The number of neighbors.	{1,**5**,9,13,17}
	Rule	C11	Repeated Incremental Pruning to Produce Error Reduction (Ripper)	[N] The number of runs of optimizations.	{1,**2**,3,4,5}
		C12	Ripple Down Rules (Ridor)	[N] The number of shuffles to randomize the data in order to get better rule.	{2,6, **10**,14,18}
Regression based Pointwise Approach	Tree	R1	Decision Tree Regression (DTR)	[N] The minimum number of samples required to split an internal node.	{**2**,3,4,5,6}
		R2	Random Forest Regression (RFR)	[N] The number of trees in the forest.	{**10**,20,30,40,50}
	Linear	R3	Linear Regression (LR)	[L] Whether the regressor will be normalized before.	{true, **false**}
		R4	Ridge Regression (RR)	[N] Stop the algorithm if whas converged.	{0.1,0.01,**0.001**,0.0001,0.00001}
		R5	Least Angle Regression (LAR)	[L] Whether the regressor will be normalized before.	{**true**, false}
		R6	Genetic Programming (GP)	[C] Feasible solution space.	[-20,20]
				[N] Population size and maximal generation.	{**100**}
	Nolinear	R7	Neural Network Regression (NNR)	[N] The number of neurons in the hidden layers.	{4,8,16,32,64,**100**}
				[N] L2 penalty (regularization term) parameter.	{0, **0.0001**, 0.001, 0.01, 0.1}
		R8	Support Vector Regression (SVR)	[N] Penalty parameter of the error term.	{0.25, 0.5,**1**, 2, 4}
		R9	Relevance Vector Machine (RVM)	[N] Penalty parameter of the error term with Linear kernel.	{**1**}
		R10	Multivariate Adaptive Regression Splines (MARS)	[N] Penalty parameter of the error term. The maximum degree of interaction (Friedman's mi). The default is 1, meaning build an additive model (i.e., no interaction terms).	{**1**}
	Lazy	R11	K-nearest Neighbors Regression (KNR)	[N] The number of neighbors.	{1,**5**,9,13,17}
		R12	K*	N/A	N/A
Pairwise Approach		P1	Ranking SVM	[N] Penalty parameter of the error term with linear kernel.	{**1**}
		P2	RankBoost	[N] The number of rounds to train.	{**300**}
		P3	RankNet	[N] The number of epochs to train.	{**100**}
		P4	LambdaRank	[N] The number of epochs to train.	{**100**}
Listwise Approach		L1	ListNet	[N] The number of epochs to train.	{**1500**}
		L2	Coordinate Ascent	[N] The number of iterations to search in each direction.	{**25**}
		L3	Learning-to-Rank (LTR)	[C] Feasible solution space.	[-20,20]
				[N] Population size and maximal generation.	{**100**}

Implementation of algorithms were based on *sklearn, Weka and RankLib* and [N] denoted an empirical value, [L] denoted a logical value, and [C] denoted the continuous space. The experimentation was conducted using 49 module-level datasets for 34 algorithms and the results were evaluated using various performance measures. The results indicated that "*Random Forest Regression*" and "*Learning to Rank*" were suited for cross-release models while "*Linear Regression*" and "*Ranking Support Vector Machines*" were opted for cross-project models.

The research papers [41], [43] reveal that ensemble methods like *Gradient Boosting*, *Bagging* and *Random Forests* are the prominent focus in SDP due to their varied perspective on the ability to identify and predict defect. With the use of various performance measures such as F1-score, AUC-ROC, data-driven comparative analysis was also conducted showcasing the effectiveness of ensemble methods in SDP.

There are also various past studies as inferred in [40] that "*Random Forest*" is one of the best defect prediction methods and in accordance with the present scenario in software development, the amount of data required for testing of the model is inadequate for advanced ML. It is also supported by [9], [15], [16], [18]. where after experimentations and result analysis it suggests that ensemble learning techniques perform best when compared to other ML algorithms and produce result like the Neural network model.

Several studies propose multiple approaches for determining an effective ML approach for efficient fault identification and mitigation of future recurrences [46], [47], [48], [49]. However, the challenges include the complexity of setting up the model training data and the resource intensive nature of extraction and preprocessing, which is not possible for organisations with limited data and resource availability.

Overall, it can be understood that most of the algorithms show high rate of data dependency especially for real-world instance shows sophistication in scalability as the frameworks are domain-specific and lacks generalizability [50], [52], [53], [54] which deviates from our area of interest.

Ensemble methods such as *random forests* and *extreme gradient boosting* that provides the necessary balance as it is less resource intensive making it adaptable and scalable and with the use of hybrid algorithm such as feature selection and ML

classifiers, it was seen that they proved to be the most effective by solving the data imbalance and reduced dimensionality [55].

Moreover, with the use of a hybrid classifier such as Voting Classifier was seen to be already addressing some of the limiting factors of existing algorithms such as generalization and ability to handle data imbalance, sparsity and biases. The article [56] states *"A voting classifier is a machine learning model that gains experience by training on a collection of several models and forecasts an output (class) based on the class with the highest likelihood of becoming the output"* which have the potential to be favourable model for organisations with limited data and resource availability.

Main **conclusions** after the literature review in this chapter are the following:

1. Traditional RCA methods provide structured procedures but are limited to biases, linear analysis and a lack of scalability resulting them in effective for sophisticated software systems.
2. Automated, ML-driven RCA approaches can overcome these limiting factors by enhancing RCA processes, leverages scalability and flexibility in dynamic development environments.
3. ML-based SDP is critical for RCA because it facilitates proactive defect identification and optimise resource utilization by identifying possible defects.
4. Challenges exist in attaining generalisability and reducing data dependency in ML-based RCA, especially for businesses with confined data availability.
5. Ensemble ML models like *Random Forest* and *Extreme Gradient Boosting* are among the most successful RCA showing close resemblance to *Neural Network* results while promoting versatility, data management.
6. MA of contextual data and leveraging the ML capabilities, the limiting factors for businesses with limited data availability can be addressed.

To establish practicality and effectiveness of ML-based RCA model and with the employment of algorithms mentioned previously, the author will build an experimental model based on ensemble ML-model leveraging the MA using contextual defect data for detecting the root causes discovered during software testing.

2. METADATA EXTRACTION AND DATA PREPERATION

In this chapter of thesis, the author discusses about the extraction process of contextual data and it's preprocessing to produce the necessary training, testing and validation dataset for the proposed ML-based RCA model, and additionally to have a reliable, anonymised data for the model development task in Chapter 3.

As a result, this chapter is split into several sections. In section 2.1, the metadata extraction process is explained. The author extends the metadata extraction source along with the use tools such as API platform for data collection, the hosted service for data merging. Section 2.2. provides the in-detail approach used for the data preprocessing, cleaning and anonymising to prepare a high-quality training and testing dataset for the RCA model in Chapter 3.

2.1 Metadata extraction

2.1.1 Selection of project management software

To achieve the experimental objective, the author of this thesis commenced with the initial step, data extraction. The scope of this study, as defined earlier, is specifically for the organisations with constrained data availability. Businesses operating in today's IT sphere, almost all, utilize the project management software app to boost the efficiency, transparency and productivity of the team, aiming to improve the overall project quality [57].

The article, [57] further states that JIRA as "*Best suited for software development teams that follow agile methodologies. Jira supports scrum, Kanban, and integrates well with code development environments.*"

In a recent survey conducted by Digital.ai, the respondents' answers to when asked about the methodology they were utilizing and in its 16th State of Agile report, also adhered in the article [58] show that the most popular methodology used was Scrum with a staggering 87%.

The respondents were further asked about their working sector to which 27% of the respondents reported to be in the technology followed by financial services that makes about 18%. This is visualization of this data is represented in figure 2.1.

Further, to satisfy requirements of the agile software development methodology as well as noting the report statistics, JIRA is chosen as it is one of the best agile

management tools for issue tracking, aligning with cross-functional teams, backlog management, project tracking and reporting along with the possibility to automate task and workflow [59] .

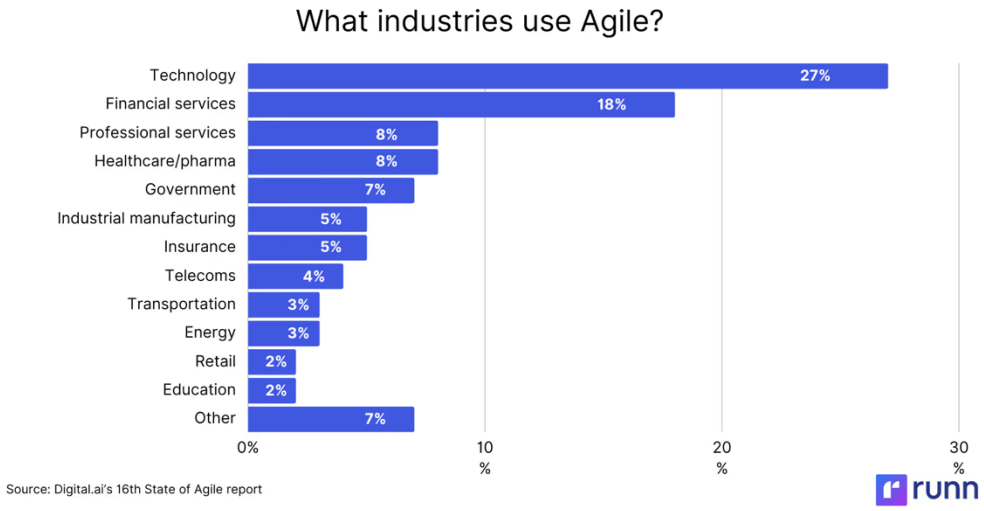

Fig. 2.1. Respondents work sector visualization (adopted from [58])

Hence, from the various spectrum of the agile project management tools, the author selects JIRA to conduct our metadata extraction which forms the base of this research.

2.1.2 Data extraction

With the project management software selected, the author selects the use of POSTMAN for the tickets metadata extraction. The metadata extracted, to be used for the experimental part of this thesis, is from two real-world projects which uses JIRA as their project management tool for issue tracking and reporting and author has the consent if sensitive data was anonymised showing no relation to the projects and to their clients, which is later done in Section 2.2.

For the data extraction process, the author creates an API token for communication between services using REST API.

The author of this thesis uses synchronous GET requests for invoking service API of which the syntax is provided in table 2.1.

Table 2.1 Syntax used for GET request

Method	Path	Description
GET	/rest/api/3/search	Query tickets using JQL

Further, the author provides the project name along with the specific number of tickets, which is 100, to be fetched at each response without compromising the data reliability [60]. The syntax full syntax is provided as:

https://example.atlassian.net/rest/api/3/search?jql=project=MYPROJECT1&maxResults=100

https://example.atlassian.net/rest/api/3/search?jql=project=MYPROJECT2&maxResults=100

The query parameters for the above synchronous GET requests are provided in table 2.2.

Table 2.2 Query parameters

Method	Type	Description	Value
jql	String	JIRA query language for filtering issues	project = MYPROJECT1/2
maxResults	Integer	Maximum number of tickets to fetch	100; to prevent server overload
startAt	Integer	Start index; pagination approach	Incremented by 100 for each request

The author incrementally increased the value of maxResults by 100 to validate the fetched data reliability and ensuring data retrieval from JIRA API's constraints.

A total of 48 GET request were sent and the responses for each were saved. Each of the responses (JSON) were appended using Google Colab.

The total number of tickets present for both projects combined, as of 04.11.2024, were 4827 showing similarity with the JSON fetched responses. The appended dataframe had a total of 18 columns which is represented in table 2.3.

Table 2.3 Dataframe attributes (version 1)

Attribute name	Type	Description	Example
Ticket_ID	Object	Unique identifier	1001
Ticket_Key	Object	Ticket key	PROJ-1001
Summary	Object	Short description	
Created_Date	Object	Date and time when the ticket was created	2024-09-30T14:49:36.052+0300
Description_Content	Object	Detailed description	
Labels	Object	Tags associated	UI/UX
Environment	Object	Environment where the issue was traced	Chrome
Components	Object	Affected services or modules	Frontend
Status_Name	Object	Status of the issue	Done
Status_Description	Object	Explanation of the status	The issue has been resolved
StatusCategory_Name	Object	High level ticket status	Fixed
Priority_Name	Object	Severity	Critical
Resolution_Name	Object	Final resolution	Fixed
Reporter_DisplayName	Object	Reporter name	Naruto
Reporter_TimeZone	Object	Time zone of the reporter	UTC-5 (New York)
Assignee_DisplayName	Object	Assigned person name	Itachi
Assignee_TimeZone	Object	Time zone of the assignee	UTC1+1 (London)
Comments	Object	Comments added	Ticket can be closed

This summarizes the study's data extraction process from JIRA using API postman whose JSON formatted responses were appended and transformed into CSV format for commencing the data preprocessing.

2.2 Data Preparation

To ensure the data reliability, integrity and observability, it is crucial to preprocess and transform the fetched data as it raw and noisy, incomplete, incorrect and contains irrelevant information. Hence, data preprocessing is a crucial step to metadata attributes to clean and standardize for creating the training and testing dataset of high quality [61].

The author uses several NLP techniques to the Summary attribute for removing punctuations, URLs, non-alphabetical characters and stop words.

After successful information retrieval, author of this thesis follows a preprocessing path to in achieving the desired dataset where finally, it is split into 3 portions training (70%), testing (10%) and validation (20%) of the model [48].

The author of this thesis segments this chapter into two sub-section 2.2.1, where the data is standardized and 2.2.2, where the data is cleaned and anonymised. A total number of 5 versions were created, each showing progression towards cleaning and anonymizing. The final version of the code is added in appendix 3.

2.2.1 Metadata standardization

A major challenge during information retrieval is standardizing due to their structure and content being inconclusive, unclear, nested or incomplete. The recovering of the metadata structure and its content is critical for effective data integration, search and analysis and which in turn enhances the data quality and decision making [62].

The author uses Google Colab, which is a free, cloud-based environment for coding, executing and ease in sharing .ipynb or .py files with Google's ecosystem through an interactive web interface.

A total of 18 attributes were fetched as shown in table 2.3, the author begins with the initial step of empty value handling after replacing all the NaN values of the dataframe with an empty space.

```
data.fillna('', inplace=True)
```

After all NaN values of the dataframe were replaced with an empty space, a total of 8 irrelevant columns, that showed no linkage with the study, were dropped by the author.

Further after dropping the irrelevant columns, there were a total of 12 columns which contained object datatypes, are shown in table 2.4.

Table 2.4 Dataframe attributes (version 2)

Attribute name	Type	Description	Example
Ticket_ID	Object	Unique identifier	1001
Ticket_Key	Object	Ticket key	PROJ-1001
Summary	Object	Short description	
Created_Date	Object	Date and time when the ticket was created	2024-09-30T14:49:36.052+0300
Description_Content	Object	Detailed description	
Labels	Object	Tags associated	UI/UX
Environment	Object	Environment where the issue was traced	Chrome
Components	Object	Affected services or modules	Frontend
Status_Name	Object	Status of the issue	Done
Status_Description	Object	Explanation of the status	The issue has been resolved
StatusCategory_Name	Object	High level ticket status	Fixed
Priority_Name	Object	Severity	Critical
Comments	Object	Comments added	Ticket can be closed

To promote effective analysis in later phase of the study, the author of this thesis aims to properly format the *Created_Date* attribute. The desired structure is to remove the timestamp and fetch only the date in the order yyyy.mm.dd.

For the necessary splitting and retrieval of just the date, the following code was used by the author.

```
df['Created_Date'] = pd.to_datetime(df['Created_Date'], errors='coerce', utc=True)

df['Created_Date'] = df['Created_Date'].dt.date.astype(str)
```

The use of python libraries spacy and re commences the utilization of data preprocessing techniques by the author.

a. Regular Expression (re)

This module provides the regular expression matching, it can be used both for both Unicode strings (str) as well as for 8-bit strings (byte). However, both the pattern and the search string must be of same type [63].

The author uses this library inside the clean_text (text) function for removing URLs, if present in the *Summary* attribute.

b. spaCy

This is an NPL library which is an open source and provides variety of practical tools used intensively for text processing. The research paper [64] illustrates its models as *"Their models have emerged as the de facto standard for practical NLP due to their speed, robustness and close to state-of-the-art performance."* The author uses spaCy also due to its sentence detection process to be applied for the *Summary* attribute.

As per the article [65] suggests that spaCy facilitates to divide a text into linguistically meaningful units which can be used when processing text to perform task such Named Entity Recognition (NER).

They contain two core released packages: *en_core_sci_sm*, which have a smaller vocabulary and does not support word vectors and *en_core_sci_md*, which have bigger vocabulary, word vectors inclusive. The vocabulary statistics for these two are shown in the table 2.5.

Table 2.5 Model attributes (adopted from [64]**)**

Model	**Vocabulary Size**	**Vector Count**	**Minimum Word Frequency**	**Minimum Doc Frequency**
en_core_sci_sm	59,338	0	50	5
en_core_sci_md	101,678	98,131	20	5

The author uses this library inside two functions, i.e., clean_text () and extract_information () for breaking the text into tokens using Tokenization, converting words to base forms using Lemmatization, removing coming words using Stop-word Removal, to extract entities using NER and extracting contextual relationships using

Dependency Parsing [61], [64], [66] to standardize the metadata and later to extract and validate the information from the "Summary" attribute by the author of this thesis.

To begin with, the two libraries are called by the author and the *en_core_sci_sm* model was used for NER and Dependency Parsing.

```
import re
import spacy

nlp = spacy.load("en_core_web_sm")
```

Clean_text () function is defined that uses re library for removing URLs and the NER and Dependency Parsing. It further uses Stop words and Lemmatization in order to tokenize and extract information based on some common keywords, given manually, inside the extract_information () function.

```
def clean_text(text):
  text = re.sub(r'http\S+|www\S+', 'URL', text)
  doc = nlp(text)
  tokens = [token.lemma_ for token in doc if token.is_alpha and not token.is_stop]

  cleaned_text = ' '.join(tokens)
  return cleaned_text
```

Manual keywords were provided by the author that would show show relevance, to the common bug description for both cause_keywords and effect_keywords such as error, issue, timeout, failed, blocked, missing etc., inside the created extract_information () function.

```
def extract_information(doc):
  cause_keywords = ['keywords were given manually']
  effect_keywords = ['keywords were given manually']

  cause = []
  effect = []

  for e in doc.ents:
    if e.label_ in ['ORG', 'PRODUCT', 'GPE', 'LOC']:
        cause.append(e.text)
    elif e.label_ in ['EVENT', 'WORK_OF_ART', 'MONEY']:
        effect.append(e.text)
```

The author then processes the extracted information to gain contextual understanding and verb identification which could indicate causes and effects along with checking the relationship between nouns and verbs.

```
    for token in doc:
        if token.dep_ == 'ROOT':
            if token.lemma_ in cause_keywords:
                cause.append(token.text)
            elif token.lemma_ in effect_keywords:
                effect.append(token.text)

    for token in doc:
        if token.dep_ == 'dobj' and token.head.dep_ == 'ROOT':
            if token.text.lower() in effect_keywords:
                effect.append(token.text)

    return ' '.join(set(cause)), ' '.join(set(effect))
```

Finally, the author extracts and validates the information from the "Summary" Column through manual inspection. It was seen that two new columns namely "cause" and "effect" were added to the initial dataframe and the newly added "cause" attribute showed linkage, to some extend, with the source of the issue.

```
df['cleaned_summary'] = df['Summary'].apply(lambda x: clean_text(x))

df[['cause', 'effect']] = df['cleaned_summary'].apply(lambda x:
extract_information(nlp(x))).apply(pd.Series)

summary_table = df[['Summary', 'cause', 'effect']]
```

The pre-existing "Label" attribute's values were over-written with the values from the "cause" attribute. It was seen that the "Description_Content" and "Comments" attributes contained nested JSON structure and upon manual investigation and with the use of further libraries, it was dropped as majority of the information showed relevance to the content already pre-existing in the "Summary" column and few showed the need for advanced data preprocessing techniques which is out of scope of this study.

Moreover, for the "Comments" attribute, it was seen that the shared information was in .png format, which also was out of context from this study. The second and this version of our dataset was created up to this point.

2.2.2 Metadata cleaning

With the creation of the “Labels” attribute using data preprocessing technique, manual verification and validation of its values were needed to ensure consistency and reliability along with maintaining the context with the “Summary” attribute.

A total of 72 unique labels were seen, of which contained labels with several typos and inconsistencies. The following code was used to fetch the number of unique labels along the label name itself from the attribute “Labels” inside our dataframe:

```
all_labels = dataframe['Labels'].str.split(',').explode().str.strip()

unique_labels = all_labels.unique()
num_unique_labels = len(unique_labels)

print("Number of Unique labels:", num_unique_labels)
print("Unique labels:", unique_labels)
```

To address this issue, the author of this thesis drops the indexing and addresses combined labels through white-space removal, corrections for handling standardized labels and then sorting the final attribute values for better analysis. It was seen that the total of 40 unique labels were seen under the “Label” attribute. However, by using the .explode () function to normalize the data present under the “Labels” attribute, the number of rows were increased to 6919 from the initial 4814.

The following code with rule-based approach was used by the author that facilitated removal of the rows when all the values under the attributes “Created_Date”, “Summary” and “Status_Name” were identical.

```
original_shape = df_reset.shape
df_reset = df_reset.drop_duplicates(subset=['Ticket_ID', 'Created_Date', 'Summary','Status_Name'],
keep='first')
```

It was seen that the rows were restored to its original count along with the updated unique label count of 40 for the “Labels” attribute and empty rows check.

Further, the values under the “Status_Name” were also analysed, cleaned and updated by the author which had a total of 12 unique labels instead of 21, as shown in table 2.6.

Table 2.6 Unique labels under Status_Name attribute

Status_Name (before)	**Status_Name (after)**
Done	Done
Log time	Log time
New	New
Closed	
On Hold	
Client Action	Client Action
Ready To Do	
QA on UAT	QA on UAT
In Progress	In Progress
QA on LIVE (migrated)	QA on LIVE
Ready for development	Ready for development
Failed QA	Failed QA
QA on DEV	QA on Staging
Client Acceptance Of The Done Work	
Scoping	
Release to Staging	
QA on Staging	QA on Staging
Release to Live (migrated)	Release to Live
PM Action (migrated)	PM Action
Release to DEV	
Code review	

The author performs this step of data manipulation to remove any labels that could lead to redundancy, inconsistency and ambiguity while conducting the root cause analysis during the later part of this study. As the “QA on DEV” and “QA on Staging” difference was just based on the instance type it was deployed (different instance for two projects), the author of this thesis, changes the label name from QA on DEV to QA on Staging for providing data consistency and integrity.

2.2.3 Analysis of system parts under development for vulnerability

To create a testing and training dataset of high-quality and to effectively conduct our root cause analysis in the later stages of this study. To achieve this, there are three general properties to be considered for our dataset and they are as follows:

1) **Elements classifications**

It aims in classifying the source of the issue, which in our case is given under "Labels" attribute. However, to form a holistic approach, the need to classify the values into either of the best-suited source is required i.e., frontend and backend.

2) **Analysis of task description**

It aims in understanding the potential cause for the ticket creation, to which the attribute "Status_Name" provides a direction to certain extend.

3) **Analysis of user stories**

To analyse the user stories is out of scope for this study due to access permissions, hence, the author of this study utilizes the "Status_Name" for drawing effective conclusions about the user stories.

Elements classifications

For effective classification of the issues to either frontend or backend. Initially, the author provides a general set of keywords for classification of known labels. The general, manually given dictionary by the author is represented in table 2.7.

Table 2.7 Keyword dictionary (Elements attribute)

Elements	Keywords
Frontend	UI, UX, Frontend, UI/UX, PDP, PLP, SEO, Home, Navigation, Payment, Search, Layered Navigation, Pop Up, Promotions
Backend	API, ERP, Sync, Backend, Server, Code, Database, Security, Performance
Unknown	Assigned to other labels, if keywords found were outside the described ones in the dictionary

The results of this resulted in the addition of a new attribute "Elements" into our dataframe.

With the added attribute, still the classification for "Unknown" labels was to be done and this was done by the author with the use of transformers as the linguistic analysis capability of this library was seen to be the most effective when compared with Natural Language Toolkit (NLTK) and spaCy as described in [67]. Among several Natural Language Processing (NLP) libraries, the article further defines it as "*The Future of NLP.*"

Transformer is a deep learning architecture developed by Google researchers and provides powerful models such as BERT, GPT, RoBERTa and DistilBERT used for text classification, sentiment analysis and topic modelling [68].

As [69] BART need in pretraining sequence-to-sequence models which pre-trains a model combining Bidirectional and Auto-Regressive Transformers. Further, the article [57] shares *"BART is particularly effective when fine tuned for text generation but also works well for comprehension tasks."*

The author of this thesis uses the BART model to do zero-shot classification without explicitly training on specific labels and which explains the link between the text and probable labels, which are to be filled instead of "Unknown" values for the "Labels" attribute [69], [70], [71].

The model is called by the author with the following syntax:

```
from transformers import pipeline

classifier = pipeline ("zero-shot-classification", model="facebook/bart-large-mnli")
```

The author defines the labels as "Frontend" and "Backend." A c_element () function was defined where the classification for "Frontend" and "Backend" labels and as per the model obtained results of the labels, the highest scored label is returned and the "Unknown" under the "Labels" attribute is replaced.

After the execution of the code, the "Elements" attribute had two labels, namely Frontend (2303) and Backend (2478). Later, data verification and validation were conducted by the author of the thesis.

The code is found under Appendix 3.

Analysis of task description and user stories

The task description is an essential part to understand the potential root causes of the defect, which resulted in the creation of the ticket. Further, it also sheds definitive insights, to certain extend, about the team members roles that could impact and support the ticket creation. Concisely, the author attempts to understand the potential cause for the ticket creation, to which the attribute "Status_Name" provides a direction to certain extend.

As described in table 2.9., the present version of dataframe contains 12 unique labels under the "Status_Name" attribute. The author of this thesis aims in finding a general linkage of the "Status_Name" with the task description and user-stories, which could mark as a high relevance during the RCA of the results in chapter 4.

Figure 2.2 depicts the usual software development approach that does not result in any errors, known as the happy route. Failure at any phase indicates that the program is not advanced ahead and is returned to development for repair and possibly future updates in line with the aim. If a failure occurs after development, the release is reverted to the previous stable version.

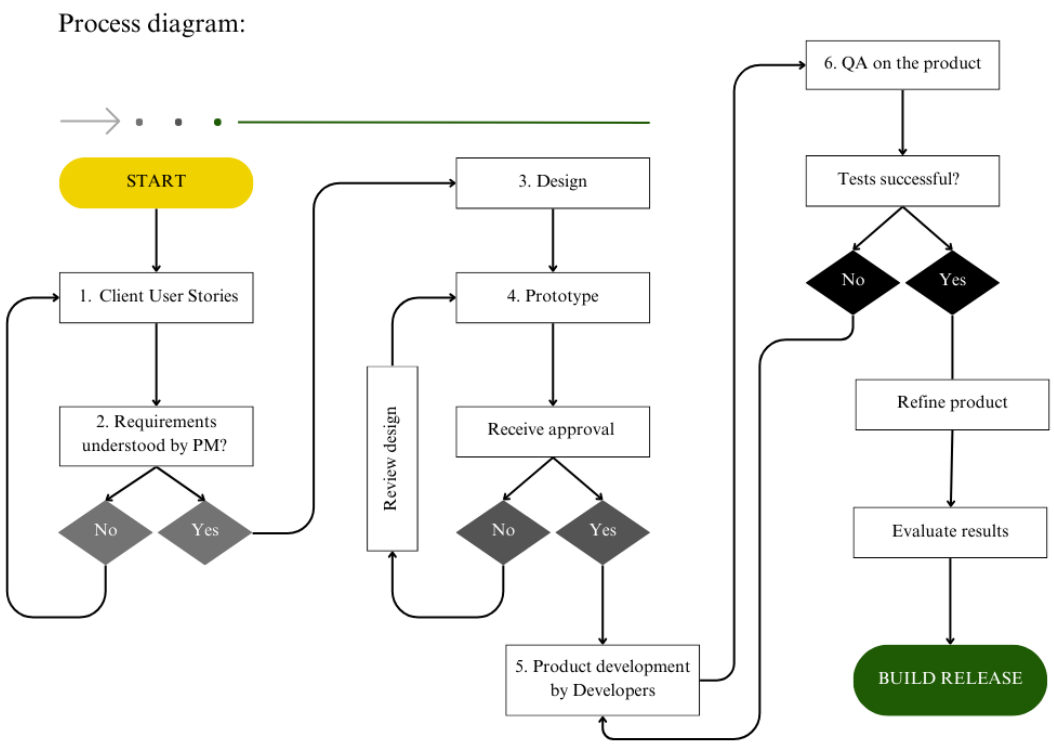

Fig. 2.2. General software development approach

The author of this thesis, in accordance with the general workflow, attempts in finding the potential points of root causes for the flagged defects through the pushed label of the extracted tickets. A new attribute "Task_Description" to the pre-existing dataframe that corresponds to 12 possible statuses for the ticket and possibility to draw meaningful insights with the help of our model, which is done later in chapter 3, in findings the potential root causes. The "Task_Description" have 12 unique labels as shown in table 2.8.

Table 2.8 Keyword dictionary (Status_Name attribute)

Status_Name Labels	**Task_Description**
Done	The ticket-specific issue has been solved
Log time	Ticket for time logging
New	Newly assigned ticket for adding or modifying specific functionality.
Client Action	Clear communication needed with the stakeholders on refining their expectations
PM Action	Re-align with the requirements and communicate with the stakeholders, if user-stories are refined
Ready for development	There were some ambiguities or anomalies observed. It can be in the requirements or in the fixes.
Failed QA	The actual and expected outcome did not meet
QA on LIVE	The responsibility on QA for ensuring active communication, if anomalies are observed in production environment
QA on UAT	The ticket is to be tested by the client facing team, before it can be deployed to the production environment
QA on Staging	The issue is in QA on the staging environment
Release to Live	The issue has been resolved and is being released to live

The final step for data preparation is fixing the data imbalance as stated in paper [6] as *"Conventional classification algorithms cannot achieve a good effect in the classification of unbalanced datasets, but they can achieve a fairly good effect in the classification of relatively balanced datasets."* This is also true as the accuracy of definitive modules is affected and limited.

The author during the dataset analysis observed that the data is not balanced, in some cases severe e.g., for "Status_Name" attribute where the highest repeating label was seen to have a total count of 4289 and the lowest was 5.

To overcome modal bias towards dominant classes and promote model effectiveness for real-world applications, the author addresses the imbalance of the dataset by the application of SMOTE (Synthetic Minority Oversampling Technique) [6][49], to ensure the model is effective and reliable for addressing critical scenarios. SMOTE was applied for 3 attributes namely "Priority_Name", "Labels" and "Status_Name." The initial unique count for the attributes is shown in table 2.9, 2.10 and 2.11 respectively.

Table 2.9 Status_Name attribute label count

Elements	**Number of keywords**
Done	4289
QA on UAT	125
New	106
QA on LIVE	86
Client Action	83
Log Time	21
QA on Staging	21
In Progress	17
Ready for development	12
Release to Live	9
Failed QA	7
PM Action	5

The author of the thesis plans on grouping of the underrepresented labels to into "Other" except for critical underrepresented labels *PM Action, QA on Staging, Failed QA and Ready for development.* Furthermore, the attribute is label-encoded for quantitative compatibility with the use of one-hot encoding.

The author also generates synthetic samples for underrepresented class for balanced training data.

Table 2.10 Labels attribute label count

Elements	**Number of keywords**
BE Admin Level	749

Checkout	698
UI/UX	317
CMS	233
PLP	186
Upgrade	181
PDP	173
API	173
Performance	159
ERP	157
Internal	154
Code	153
CART	150
SEO	132
Home	119
URL Migration	108
Maintenance	96
Search	95
Promotions	91
Payment	82
Cross Device	81
Pop Up	78
Footer	55
Shipping Address	51
Sync	42
CRON	35
Shipping Method	33
Broken Links	31
Sign In	28
My Account	25
Sitemap	21
Downtime	17
Layered Navigation	15
Cross Browser	14
Security	10

Navigation	10
Newsletter	10
Conversion	8
Create Account	6
Usability	5

Similar approach of representation of underrepresented labels (less than 50 occurrences) are used by the author, terming them as "Other" for avoiding bias and promote model efficiency and reliability. Later, it was label-encoded to quantitative values with the use of one-hot encoding.

Table 2.11 Priority_Name label count

Elements	**Number of keywords**
High	1865
Medium	1804
Low	705
Critical	276
Blocker 24/7	131

Similar approach was used for this attribute by the author and finally with the help of SMOTE, dynamic adjustment of k_neighbors parameter (based on the size of the smallest class was used and the labels were assigned numerical values.

The new attributes were added to the dataframe as "Status_Name_grouped", Labels_combined and "Priority_Name_encoded." The fourth version of the dataset was created by the author which still implied the need for data anonymization to respect the client's and projects need for privacy.

This is addressed by the author by extracting the total number of unique keywords from "Summary" attribute (3098) and anonymising the sensitive keywords using re library to be used for training and testing of the ML-Based RCA model in chapter 3.

3. PROPOSED ML-BASED RCA MODEL

In this chapter, the author builds the RCA model for finding the potential root cause of defects discovered during software testing through metadata analysis and leveraging machine learning algorithms. Section 3.1 deals the existing approaches gap, the unsuitability of the existing methods, Section 3.2 deals with selection of the features and targets and the concept being used along with the schema. Section 3.3 deals with model selection, Section 3.4 deals with the Training pipeline that consists of model training and evaluation and in Section 3.5, the research questions are realised by the author.

3.1 Related work

Presently, the existing approaches to root causes analysis is diversified as it includes manual RCA, rule-based automation and statistical methods. However, all the methods are tedious, subject to biases, lack generalization and often struggle to evolve to the dynamic needs and trends. Papers [9], [48], [72] suggest that the ensemble algorithms due to its tuning and boosting capabilities prove to have an edge over this case while the ability to detect the multi-faceted relations in both structured and unstructured data.

Experiments conducted in literature reviews suggests the effectiveness of the ensemble approaches in manging the class imbalances (underfitting and/or overfitting) and producing consistent and reliable outcomes [9], [41], [45], [72] while also showcasing the performance improvement with the availability of boosting and hyperparameter tuning when done strategically. However, it was also noted in the literature review that the authors prompted in the selection of hybrid algorithm to overcome rigidness and ensure generalization.

Voting Classifier is opted by the author of this thesis for empirical evaluation of the models to be done in later stages of this chapter with the advantageous characteristics of this algorithm as discussed in article [56], [72].

3.2 Features and target selection

In this section, the author of this thesis, in accordance with the scope, the features were chosen based on their relevane for SDP and 3 different feature types were taken into consideration which comprises both structured and unstructured data for increased reliability of the model to capture the pertaining defect characteristics.

1. **Metadata**

   The author selects two of the attributes available in our dataset namely *"Elements"* and *"Labels"* were taken into consideration as it aids in determining the scope and impact.

2. **Contextual**

   The author has chosen the two attributes namely *"Summary"* and *"Task_Description"* for feature extraction using SentenceTransformers to understand the semantic relationships and the potential causes of the defects as well as the origin of the defect or its affects.

3. **Pre-processed**

   The newly added attributes namely, *"Priority_Name_encoded", "Status_Name_grouped"* and *"Labels_combined"* are chosen by the author of the study for optimal model training and eradicate or reduce sparsity in results.

Regarding to the Target selection, the author of this study has selected three of the attributes namely*, "Priority_Name", "Status_Name" and "Labels"* for models with no hyperparameter tuning and *"Labels"* for models with hyperparameter tuning making use of Term Frequency-Inverse Document Frequency (TF-IDF) and a choice between RandomizedSearchCV and GridSearchCV for better contextual data handling and understanding [48], [73], [74] while also understanding the computational cost using the stated. After the features and target selection, the proper selection of the model from a wide spectrum is vital to promote and propel the study towards the main goal of the study.

The figure 3.1 shows a general schema illustrating the relation of metadata.

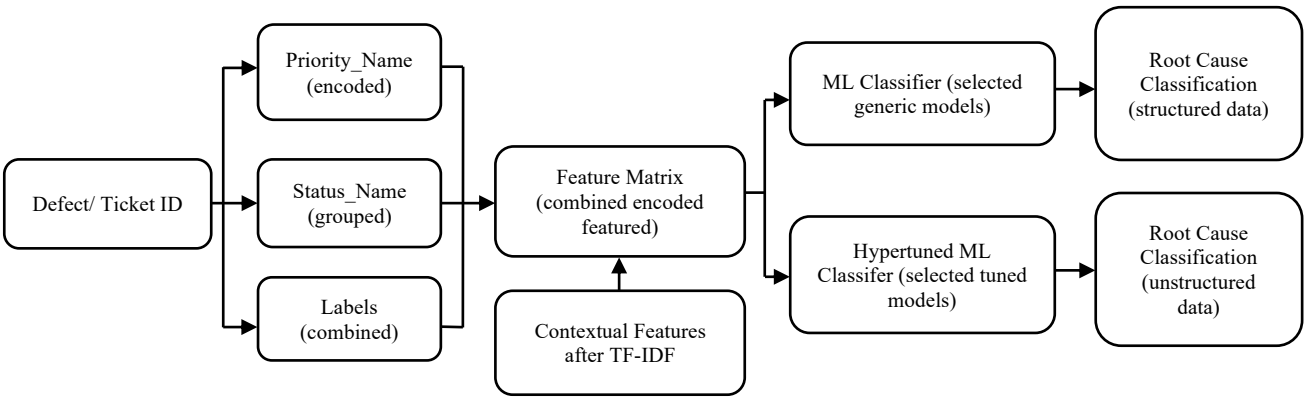

Fig. 3.1. Metadata relationship schema

Figure 3.1 illustrates the flow of both structured and unstructured metadata through the feature-engineering process and eventually fed into different ML models as an input for root cause classification. The arrows represent the projected path for the data to flow through the pipeline.

1. *Defect/ Ticket ID*

It is taken as a raw identifier, initially as JSON which later is transformed into dataframe, is fetched from JIRA through a series of API requests using Postman. It contains both the structured and unstructured data format along with high noises for each specific tickets.

2. *Structured Data attributes*

It comprises of three attributes derived from **Priority_Name, Status_Name** and **Labels**. The author encodes and groups these attributes for making the attributes more traceable and meaningful categories for capturing the boarder scope and impact of the defect.

3. *Unstructured attributes*

It comprises of the textual attributes namely **Summary** and **Task_Description** which is processed with SentenceTransformers and TF-IDF for proper contextual handling and understanding. These are then appended together into one.

4. *ML classifiers*
    a. The pre-processed data, from the *Feature Matrix*, is the input for the **generic** models (standard classifier without hyperparameter tuning). It is based on structured data only.
    b. For the attributes comprising of richer-contextual data, the author of this study used the unstructured data to be fed as an input for the **tuned** models (also *hypertuned ML Classifiers* using *TF-IDF* and *RandomizedSearchCV*) for effective contextual data handling and understanding.

5. *Outputs*

The outputs from both the ML based RCA models, for both structured and unstructured data taken separately to mitigate data sparsity and bias and to ensure reliable root cause classifications promoting greater accuracy and insights.

3.3. Model selection

3.3.1 Proposed learning model

At this stage of the study, the selection of model for training and testing on the pre-pre-processed dataset is done. Selection of a definitive model from a wide range of options in accordance with the type of dataset could be crumblesome. The papers [2], [5], [10], [75] provides various statistics, their experimental results based on which the author selects the top performing algorithms. However, before initiating the training pipeline. Referencing with the study [10], the basic RCA model workflow is given in figure 3.2.

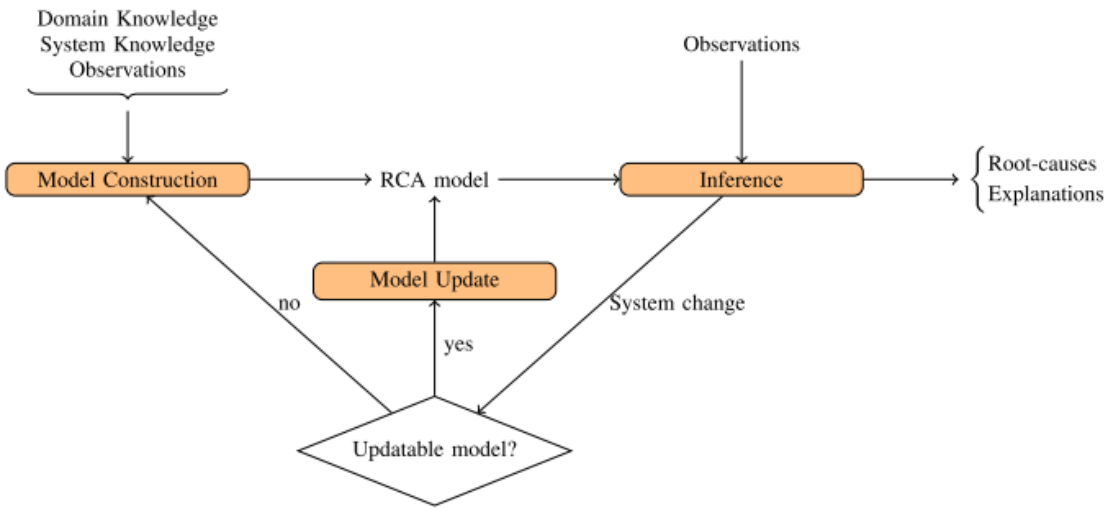

Fig. 3.2. RCA analysis workflow (adopted from [10]**)**

The study [10] describes the figure 3.2 as the summarization of the general RCA technique workflow with first the model being constructed by co-joining the Domain and system knowledge along with the observations of the diagnosed system.

Inference is seen as the fundamental task in Artificial Intelligence (AI), hence majority of the RCA models (Probabilistic and Deterministic) come from this field. The study further illustrates through its experimental analysis that how advanced classifiers such as Neural Nets, show difficulties in grasping as their possibility to forecast root causes, as they lack principles and only rely on single label categorisation. The study concludes in discussing about the importance of multi-class classifier to conduct multiple faceted RCA and end with, *"there are usual trade-off between tractability and expressiveness."*

Through the analysis of the knowledge gained from the literature review, the author of the thesis selects Support Vector Machines (SVM), Logistic Regression (LR), Random Forest (RF), Extreme Gradient Boosting (XGBoost) and the *Ensemble Learning* with *Voting Classifier* that combines the prediction of multiple models for ensuring, reliability and efficiency [56], [72].

3.3.2 Evaluation metrics

For finding the proposed model's reliability, consistency and performance, the author selects 4 evaluation measure namely accuracy, precision, recall and F1-Score which were used in majority of the research papers [8], [9], [48],[76].

The paper [8] provides the representation of the top 10 evaluation metrics obtained through analysis of more than 132 papers as shown in figure 3.3.

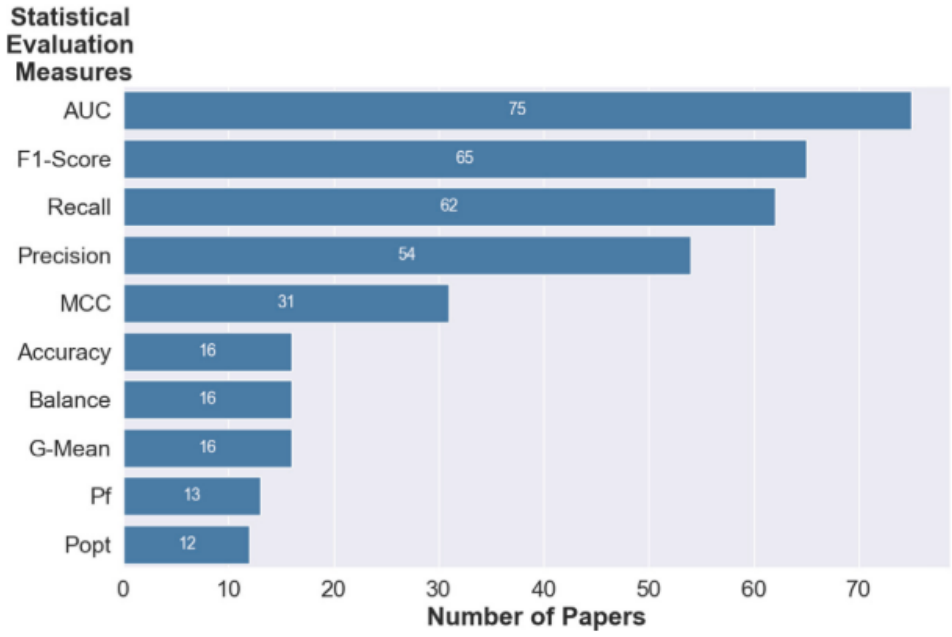

Fig. 3.3. Top 10 evaluation metrics (adopted from [8]**)**

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN} \quad (1)$$

Where *True Negative (TN)* - number of negative instances labelled correctly;

True Positive (TP) – number of positive instances labelled correctly;

False Positive (FP) - number negative instances labelled as positive;

False Negative (FN) – number of positive instances labelled as negative;

$$Precision = \frac{TP}{TP+FP} \quad (2)$$

Where *True Positive (TP)* - number of positive instances labelled correctly;
False Positive (FP) - number negative instances labelled as positive;

$$Recall = \frac{TP}{TP+FN} \quad (3)$$

Where *True Positive (TP)* - number of positive instances labelled correctly;
False Negative (FN) - is the number of positive instances labelled as negative;

$$F1 - Score = 2 * \frac{Precision*Recall}{Precision+Recall} \quad (4)$$

Where *Precision* – as defined in 1*;*
Recall – as defined in 3*;*

$$False\ Positive\ Rate\ (FPR) = \frac{FP}{FP+TN} \quad (5)$$

Where *False Positive (FP)* – number negative instances labelled as positive;
True Negative (TN) – number of negative instances labelled correctly;

$$Area\ Under\ the\ Curve\ (AUC) = \int_0^1 TPR\ (FPR)d(FPR) \quad (6)$$

Where *FRP* – as defined in 5*;*
TPR (Recall) – as defined in 3*;*

3.4. Model training

The author of this thesis before initiating the selected model training, initiated with the use of SentenceTransformer model specifically *all-MiniLM-L6-v2* to find the contextual data semantic relationships [66], [67], [77] between the attributes "Task_Description" and "Summary." The syntax for initializing:

```
model = SentenceTransformer('all-MiniLM-L6-v2')
```

For splitting the original dataset for training, testing and validation dataset. the newly added attributes, "Summary_embeddings" and "Test_Description_embeddings" were processed and converted from object type to list type. The author of the thesis aims in using the generated embeddings (contextual datatype) using hyperparameter

optimizations for each model to enhance the understanding of contextual nuances along with the use the *AUC* and *F1-Score* assessment criteria to compare the models' performance that goes beyond accuracy.

Table 3.1 Dataset structure

	Features	**Targets**
Training Set	3346, 785	3346,
Testing Set	474, 785	961,
Validation Set	961, 785	474,

The training, testing and validation set were created as shown in table 3.1. The column count was consistent for all i.e., 785 after combining the embeddings and SMOTE processed features. It was to be noted that the dataframe were converted to NumPy array and the model was evaluated with the selected performance metrics. The models used were *Random Forest, Extreme Gradient Boosting, Logistic Regression, Support Vector Machines* and *Ensemble Learning* with *Voting Classifier.*

The author of the thesis created two training and evaluation programs for the selected models. Pre-processed and metadata type were used for training generic models without the use of hyperparameter tuning while for contextual type, models using hyperparameter tuning were used. The code is given in Appendix 2 and Appendix 3 respectively.

3.5. Analysis of model performance

The training, testing and validating of the chosen models i.e., *SVM, LR, RF, XGBoost* and *Voting Classifier,* was conducted by the author with two approaches for all 5 targets and results were compared as per the chosen evaluation metrics for each type. The models used no hyperparameter tuning for targets *Priority_Name*, *Status_Name* and *Labels*, but the author introduced hyperparameter tuning for the models when contextual data, "Summary" and "Task_Description," to effectively find insights from the contextual nuances and promote effective data handling and understanding.

The performance metrics used for **generic-models** (hereinafter) when using pre-processed and metadata were *accuracy, precision, recall* and *F1-score*. For the **tuned-models** (hereinafter) introduced with hyperparameter tuning, the performance metrics were *ROC-AUC* and *F1-score*.

The author of the study selected 3 targets i.e., *Priority_Name*, *Status_Name* and *Labels* for generic-models and 2 features i.e., *Summary* and *Task_Description* with a common target *"Labels"* for tuned-models.

1. **Generic Models**

After the models were trained, tested and validated. The chosen models were evaluated based on their performance. It was seen by the author that *XGBoost* and *Voting Classifier* performance were almost perfect. The table 3.2 and 3.3 represents the models' performances during testing and validation.

Table 3.2 Generic model performance (testing dataset)

Models	**Accuracy**	**Precision**	**Recall**	**F1-Score**
RF	0.9909	0.9946	0.9909	0.9904
XGBoost	0.9979	0.9985	0.9979	0.9982
LR	0.9705	0.9635	0.9705	0.9656
SVM	0.8797	0.8255	0.8797	0.8801
Voting Classifier	0.9972	0.9981	0.9972	0.9975

For **Priority_Name** target, *XGBoost*, *LR* and *Voting Classifier* performed the best and achieved perfect classification across all the 5 classes, as seen in table 2.11. *RF* showed slight misclassifications in the *"Medium"* class and had lower performance than *XGBoost*. *SVM* faced complications, and notable misclassifications were seen for in the *"Critical"* and *"Blocker 24/7"* classes due to its sensitivity to class imbalance.

For **Status_Name** target, the imbalanced yet crucial data such as *"PM Action"* and *"Failed QA"* impacted the overall performance in models especially to *SVM* and *RF*. *XGBoost* and *Voting Classifier* showed effective rare classes handling due to its robust boosting and ensemble techniques. *SVM* performance was poor and consistent in case of the rare classes, notably for *"Ready for Development"* and *"Release to Live"* classes.

For **Labels** target, *XGBoost* and *Voting Classifier* showed consistency in their performance with 1.000 or 100% accuracy that showed the models ability to handle complex data relationships. It was observed that *SVM* struggled again with

classification of minority classes like *"Newsletter"* and *"Sign In."* *RF* and *LR* performance were mediocre when classification of highly similar classes.

Table 3.3 Generic model performance (validation dataset)

Models	Accuracy	Precision	Recall	F1-Score
RF	0.9931	0.9944	0.9931	0.9932
XGBoost	0.9979	0.9971	0.9979	0.9978
LR	0.9747	0.9653	0.9747	0.9685
SVM	0.8876	0.8465	0.8876	0.8903
Voting Classifier	0.9969	0.9961	0.9969	0.9965

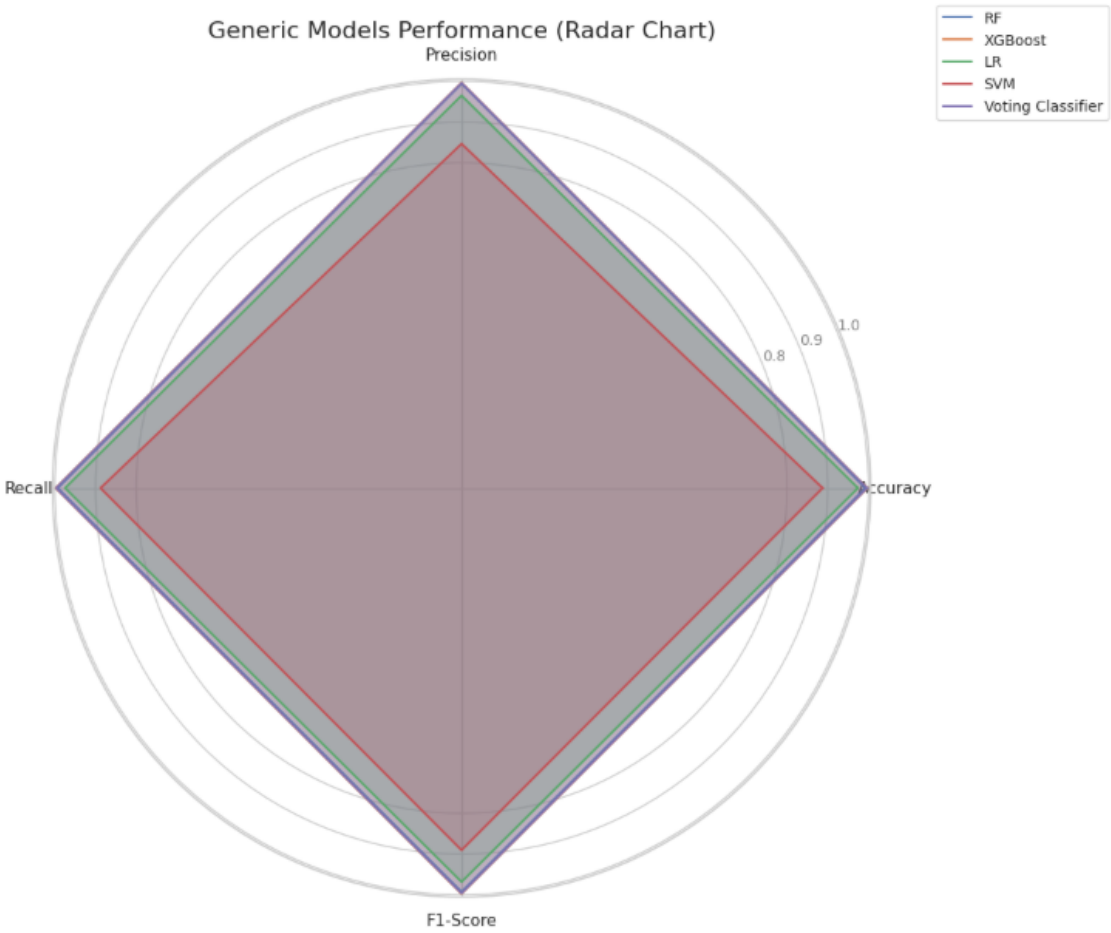

Fig. 3.4 Generic models performance

2. **Tuned Models**

The author of this thesis uses hyperparameter tuning to properly find and capture the contextual nuances from *"Summary"* and *"Task_Description"* attributes with an aim to properly handle and understand data to further propel and aid in the finding the potential root causes. The chosen models were the same, with hyperparameter tuning introduced and be fed with contextual data. The remaining 2 contextual features i.e., *Summary* and *Task_Description* were selected by the author of this thesis.

After the selection of targets, the training, testing and validation of the models were done. The key performance metrics chosen were *F1-Score* and *ROC-AUC* with the primary target chosen as *"Labels."*

The author of this thesis appends both the features into one single textual representation for models' input and which was transformed into quantitative data using the *TF-IDF* vectorizer for effective capturing of the textual nuances and effective data handling and understanding. For the search strategy *GridSearchCV* was to be used, however due to extensive use of resources, the author of this study has chosen *RandomizedSearchCV* for efficiency without compromising the performance and reliability.

Table 3.4 Tuned model performance (Validation)

Models	**F1-Score**	**ROC-AUC**
RF	0.626	0.9105
XGBoost	0.651	0.9173
LR	0.637	0.9286
SVM	0.628	0.9216
Voting Classifier	0.660	0.9374

Table 3.4 illustrates the evaluation of the tuned models, based on validation dataset, with the use of pre-defined performance metrics. It was seen that *Voting Classifier* outperformed other models and *XGBoost* was the potential substitute.

In case of *RF*, it was observed that the model performance in capturing the well-represented classes was effective but even after sampling, struggled with rare classes.

In case of *LR,* it was observed that the model showed effectiveness in general contexts but failed to understand and understand the non-linear, complex data relationships.

In case of *SVM*, it was observed that the model showed poor performance for rare classes and even though it was computationally intensive.

In case of *XGBoost*, it was observed that the model was effective in handling the imbalances and in capturing the contextual nuances with its boosting techniques but needed careful tuning and extensive training time.

In case of *Voting Classifier*, which is chosen by the author for its combined strengths from the other models and its consistent performance across all metrics and classes.

The ensemble methods are resource extensive but with strong handling, tuning and understanding the contextual nuances, it was evident that *Voting Classifier* showed balance between performance and usability that is reliable, effective and accurate.

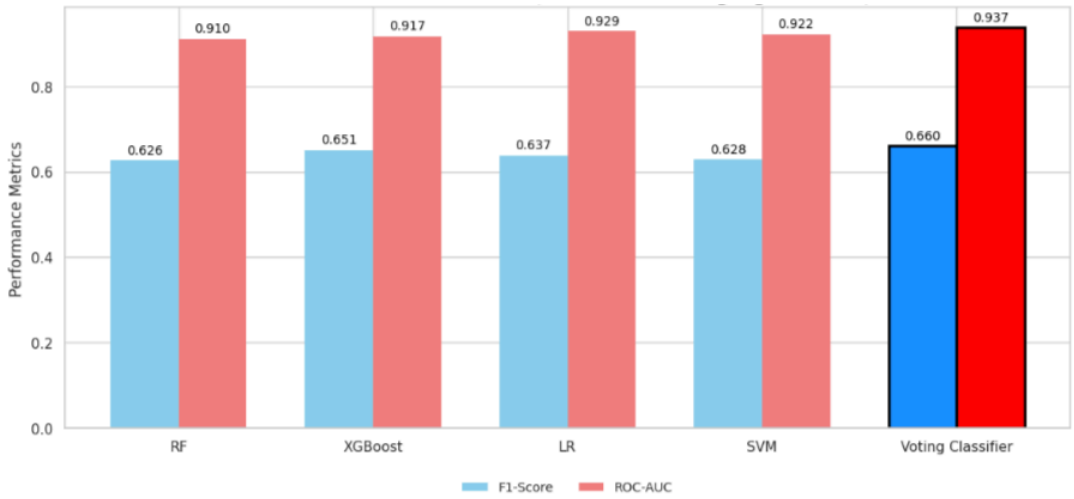

Fig. 3.5 Tuned models performance

For the **main conclusion** after the experiments and along with the knowledge from the literature review on existing algorithms and their inherent constraints [44], [47], [48], the author of this study emphasises on the use of hybrid models for the organisations with the limited resource availability and expertise, as the *Voting Classifier* was seen to be the most optimal and efficient choice portraying balance between performance and usability. The author of this study illustrates that overall improvement in generalization of the model with its capability to integrate the strengths

of multiple classifiers and the aiding computational effectiveness in mitigating the biases and weaknesses [56], [72].

Model	Accuracy	Precision	Recall	F1-Score
RF	0.9931	0.9944	0.9931	0.9932
XGBoost	0.9979	0.9971	0.9979	0.9978
LR	0.9747	0.9653	0.9747	0.9685
SVM	0.8876	0.8465	0.8876	0.8903
Voting Classifier	0.9969	0.9961	0.9969	0.9965

a)

Model	F1-Score	ROC-AUC
RF	0.626	0.910
XGBoost	0.651	0.917
LR	0.637	0.929
SVM	0.628	0.922
Voting Classifier	0.660	0.937

b)

Fig. 3.6 Performance overview: a) Generic models b) Tuned models

The overall performance of all models (in Fig. 3.6) when considered collectively, highlights the usage of *XGBoost* and *Voting Classifier* as a reliable choice for conducting comprehensive RCA.

Hence, in comparing the results from both the models, it is evident that the ensemble methods showed greater resilience and reliability throughout the experiment and with hyperparameter tuning, significant improvements for handling and performing rare, multi-classes classifications were seen. The author of this thesis also portrays the superior results making capability which makes *Voting Classifier* an effective and robust generalization model suitable for real-world scenarios.

3.5. Results and discussion

RQ1: *To what extend does the ML-based RCA model address the limitations of traditional RCA approaches?*

In training and validation, it was discovered that for targets dealing exclusively with metadata properties, the generic models (*Voting Classifier* and *XGBoost*) consistently produced near-to-perfection scores in terms of *accuracy, precision, recall,* and *F1-score* across both validation and test datasets. It empirically demonstrates that ML-based techniques can consistently achieve high-quality RCA classification, much above what traditional, manual approaches can give.

Furthermore, in uncommon classes, *XGBoost* and *Voting Classifier* got near-perfect results even without hyperparameter adjustments, demonstrating the models' superior resilience to multi-class complexities which is an acknowledged limitation in case of manual RCA. Upon introducing contextual data and hyper-parameter tuning, the Voting Classifier performed the highest which suggests that the ensemble approach performed consistently better than generic models for rare classes.

1. *Voting Classifier* (Validation) for target **Labels**: accuracy = 0.9990, precision = 0.9991, recall = 0.990 and F1-Score = 0.9989.

2. *Voting Classifier* (Tuned) for target **Labels**: ROC – AUC ± 0.94 with F1-Score of 0.66 approx.

The outcomes were inferior when contextual data was included into the model due to insufficient data and given that text classification became challenging than merely encoded information. However, research findings indicates that **ML-based RCA** (in particular, the ensemble models) **overcome typical RCA classification constraints** demonstrating comprehensiveness and reliability.

RQ2*: Limiting factors of traditional RCA and how can they be addressed using automation and ML?*

Some of the major limitations in manual RCA are the categorisation of uncommon classes, resource utilisation, and human-induced biases. Even certain models, such as *SVM*, consistently performed poorly for under-represented classes, with near-zero recall in confusion matrices. Similarly, in typical RCA, these classes may be skipped or interpreted incorrectly. However, ensemble models were able to be consistent and demonstrate perfect recall for several classes even when no modification was applied to the model.

When target labels were used for generic models, *Voting Classifier's* accuracy and F1-Score were 100%, whereas *SVM* had 0.8354 and 0.79, respectively. It is also to be observed that traditional RCA requires significant resources for 5 whys, DMAIC, Fishbone, and other tasks, whereas ensemble models achieved 100% test accuracy for the **Priority_Name** and **Labels** objectives in a single pipeline run.

Furthermore, the confusion matrices for the ML-based technique exhibited consistent trends and seldom misclassified classifications of high priority classes like *"Critical"* or *"Blocker 24/7"* which are critical for RCA which could be otherwise when

manual intervention and expertise were to be implemented. Hence, the empirical results illustrates that the **automated ML-based RCA model**, particularly the ensemble models, **reduces the known limitations of manual RCA** i.e., time requirements, biases and rare classes misrepresentation.

RQ3*: How can organisations with limited data availability and expertise, implement an efficient ML-based RCA model?*

When only the metadata features such as **Priority_Name, Labels** and **Status_Name** were employed, and the generic models already shown demonstrated outstanding calibre, with some reaching 100% accuracy. This implies that organisations in this bracket can start with just a few characteristics and still attain considerable results and performance improvements. It may be pointed out that, based on the business requirements, specific characteristics can be encoded or aggregated to minimise sparsity prior incorporating contextual data to get insights through contextual data handling and analysis: if/when further data becomes accessible. However, it was seen in the empirical results that with the introduction of contextual data, it remained strong for a multi-class textual setting.

It was observed that without the hyperparameter tuning, the F1-Score for all generic models were less than *0.40* which later was boosted to *0.66* with ROC-AUC of *0.94* in case of *Voting Classifier* when hyperparameter tuning using *RandomizedSearchCV* and *TF-IDF* were used. The more profound hyperparameter search is *GridSearchCV* but requires high computing resources are out of scope for our research.

Consequently, a predictive and efficient RCA model with generalisation may be deployed gradually, starting with the most basic structured fields and eventually including contextual data, as well as dynamically altering the ensemble models to effectively handle the insufficient data.

RQ4*: How can defect metadata be used to systematically train an ML-based RCA model?*

Pre-processed fields to address data sparsity led the models to almost 100% accuracy for generic models and with the implementation of *RandomizedSearchCV* parameter and *TF-IDF*, the contextual nuances understanding and handling were improved for tuned models.

a. Raw fields collections
b. Group or encode quantitative data to decrease data sparsity and convert text into vectors
c. Ensure balanced sampling to mitigate overfitting and/or underfitting
d. Start with preliminary models such as *RF*
e. Gradually evolve to ensemble (Voting Classifier) and boosting (XGBoost) after observing the preliminary outcomes
f. Track evaluation and confusion metrices

The figure 3.7 depicts the clear pipeline for the systematic use of metadata for promoting the active learning of the ML-based model, even with skewness or inadequate fields, from structured and unstructured data.

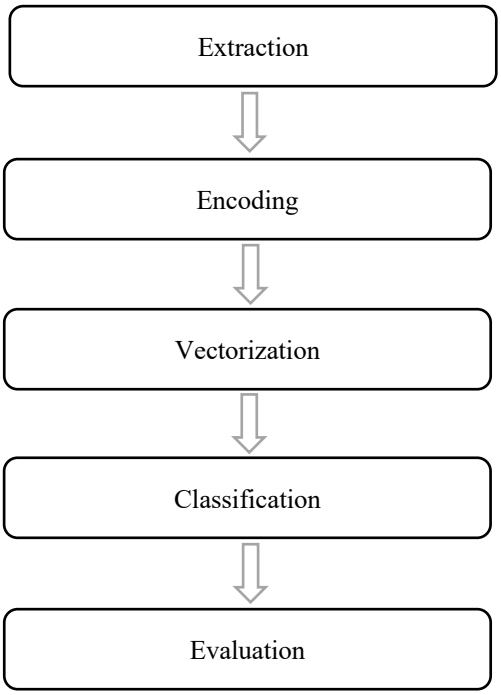

Fig. 3.7 Pipeline for effective metadata utilization

The author of this study, based on the empirical results and matrices, illustrates that the use of ML-based RCA model is significant for handling the data sparsity, biases and the resource constraints. With the ensemble techniques used incorporated with strategic handling, it was seen to produce high performance results. With the pipeline given by the author of this thesis, it assures the continual improvement, model adaptability and reliability which are the core features that the manual RCA fails to implement.

4. POTENTIAL APPLICATION OF PROPOSED MODEL

4.1 Incorporation of rule-based RCA layer

In this chapter, the author presents a rule-based approach: a potential use of the selected, trained ML-classifier model, i.e., *Voting Classifier*, that translates classification outputs into actionable root-cause insights using a rule-based guidance on.

Despite of high-level of classification capabilities, illustrated specially by the ensemble models, it is not adequate to provide aid and to drive the change in the software development and testing paradigm. The author aims to integrate layer of pre-defined set of rules to map the classified defect into potential root causes and create generalized, interpretable RCA method within the businesses.

To effectively incorporate and generate actionable RCA insights, it is essential to initially pre-define a structured taxonomy for the root causes, especially for each frequently surfaced defect label through the domain level knowledge and previous resolved and documented defect sources.

The table 4.1 illustrates an example use of pre-defined structure RCA taxonomy, for the known root causes developed with domain-level expertise and/or previously resolved and documented defect sources.

Table 4.1 Taxonomy of root causes

Elements	Potential root cause(s)	Likeable process gaps
Checkout	API Issue with the payment gateway Auto-complete logic for city and state address not working after providing Zip code	Low full regression testing frequency Gaps in integration testing
Sign In	Possibility to login using incorrect login credentials	Minimal testing coverage for corner-cases
SEO	Improper URL rewriting	Poor search engine requirements testing
ERP	Price sync mismatch between frontend and backend	Gaps in integration testing Limited monitoring

The **Status_Name** and **Priority_Name** properties can also be added to the table 4.1 for more refined and detailed hypothesis of the root causes. The tables 4.2 and 4.3 illustrates the re-defined RCA taxonomy with the introduction of these attributes.

Table 4.2 Taxonomy of root causes (version 1)

Elements	Priority_Name	Potential root cause(s)	Likeable process gaps
Checkout	Blocker 24/7	API Issue with the payment gateway Auto-complete logic for city and state address not working after providing Zip code	Rare testing for user checkout flow after deployments Poor practice of regional testing after each deployment

Table 4.3 Taxonomy of root causes (version 2)

Elements	Status_Name	Potential root cause(s)	Likeable process gaps
Checkout	PM Action	API Issue with the payment gateway Auto-complete logic for city and state address not working after providing Zip code	Lack of understanding of user-stories resulting in poor task assignment to the developers Limited functionality for the development environment

The structure of the pre-defined RCA taxonamy imposes no scalability issue and depending upon the subject matter expertise and available detailed data, the taxonomy could more business-specific or more generalised. Many merits of the rule-based approach could be drawn such as:

1. **Structured approach** as the rule-based approach relies more on subject matter expertise than on large data which makes it an ideal choice for our targeted business type.
2. **Scalability** and **maintainability** as per the organisational needs with the support of *CRUD* (Create Read Update and Delete) nature of the applied taxonomy.
3. **Interpretable** and **reliable** with the joined nature of classification accuracy of classifier and the domain-specific rule-based approach on the defect labels.

4.2 Linkage between rule-based and root causes

With the development of the structured rule-based taxonomy and the classified labels from the *Voting Classifier*, the author of the thesis illustrates the rule definition for an effective association of the classified labels with the potential root causes.

```
IF defect_label == "Checkout":
   root_causes = [
      " API issue with the payment gateway",
      “ Auto-complete logic for city and state address not working after providing Zip code”
   ]
   actions = [
      " Periodic full regression testing ”
   ]

ELIF defect_label == "Sign In":
   root_causes = [
      " Possibility to login using incorrect login credentials “
   ]
   actions = [
      " Proper testing coverage“,
      “ Testing for corner cases”
   ]

# to add/modify the rules
```

The futher fine-tuning or logic expansion is possible with use of metadata to provide deeper contextual insights. Several attributes such as *Priority_Name, Status_Name* can be considered depending upon the organisational context and complexity, conditional statements can be added or modified.

```
IF defect_label == "Checkout" AND status_name == "PM Action"
   root_causes = [
      " API issue with the payment gateway",
      “ Auto-complete logic for city and state address not working after providing Zip code”
   ]
   actions = [
      " Re-align with the requirements and communicate with the stakeholders, if user-stories are
refined”
      “ Promote active communication during the DS”
   ]

# to add/modify the rules
```

4.3 Expected outputs interpretation and recommendations

In this section, the author presents a theoratical process-flow schema that illustrates the integration of the rule-basesd layer with the ML-Classifier the expected outputs and the possible interpretation. This section ends with recommendation and conclusion drawn by the author on the overall rule-based ML classifer.

The figure 3.2 shows an updated schema illustrating the relation of metadata.

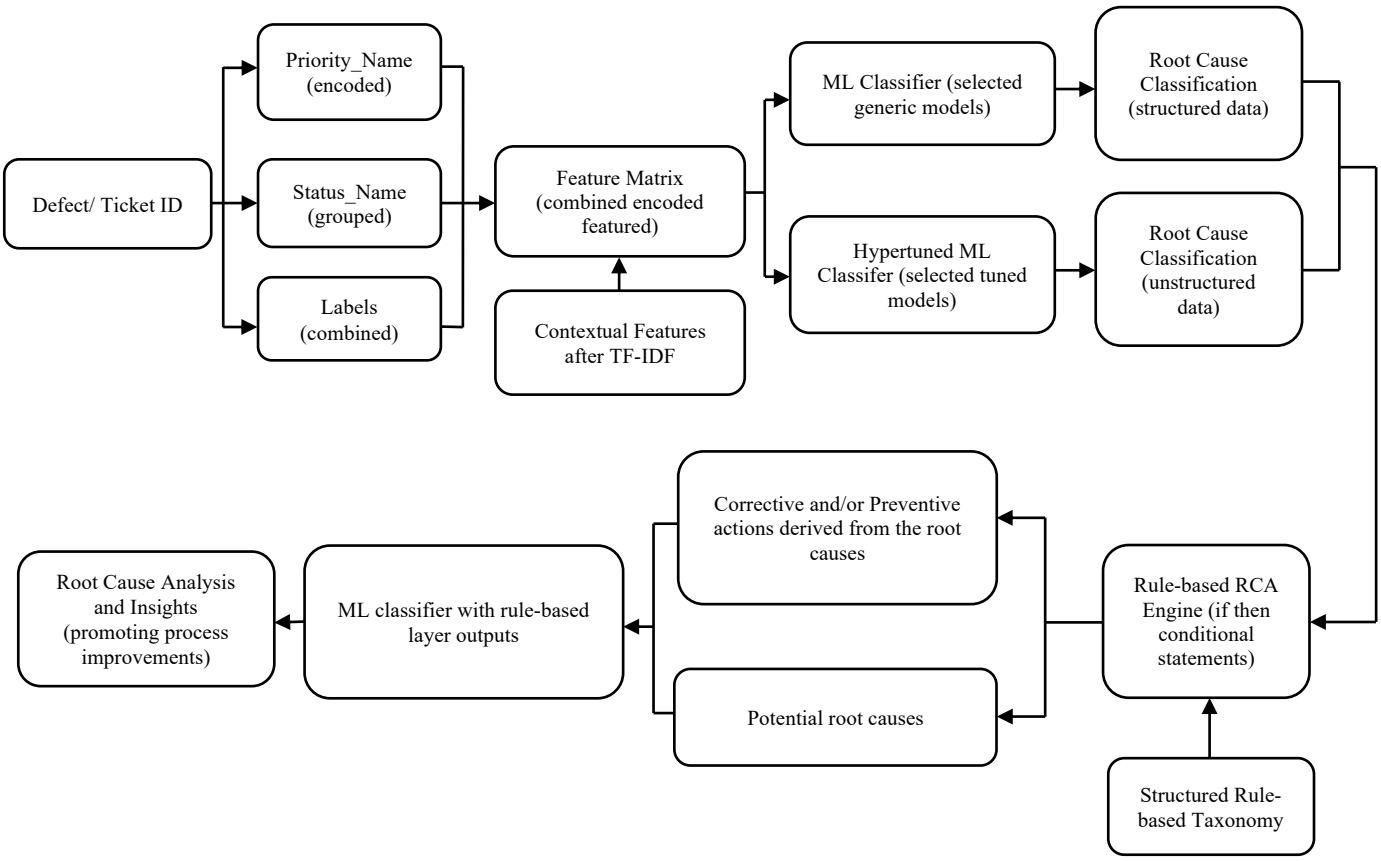

Fig. 3.2. ML classifier with rule-based layer conceptual schema

With the incorporation of rule-based layer with the ML classifier, with cross-referencing with the pre-defined RCA taxonomy and the use of conditional statements based upon the level of complexity and depth, the potential root causes are mapped and the recommended actions per classified label is given as an output. Outputs in the later stages give insights on corrective and preventive actions. Moreover, it is recommended to document the root cause data and its patterns to promote effective actionable insights and facilitate to improve rule-based taxonomy.

The schema shown in Fig. 3.2 illustrates that the defect after passing through the rule-based RCA engine, it comprises potential root causes along with the corrective and/or preventive actions. For further insights, the author of this study suggests to gradually append several defect outputs for trend data and pattern recognition with overtime, promoting for the active refinement to the structure of the rule-based taxonomy.

The **main recommendation** drawn from this chapter are:

- To inculcate the feedback loop to promote continuous improvement for both rule-based conditional statements and the classification efficacy of the chosen ML classifier.

- For the data and resource constrained organisation, the author of this study suggests initially with a rather general RCA, which still provide valuable actionable insights and then with time and data availability, to refine and incorporate process advancements.

- Initially to focus the effort on the critical areas for predicting root causes that produces the significant ROI for the business.

- To effectively implement better instrumentation for producing richer training datasets to improve ML classifier's reliability and conditional statements granularity.

Overall, with the incorporation of an additional rule-based layer to a reliable ML classifier i.e., *Voting Classifier* upon experimentation and result evaluation the potential root causes along with the corrective and/or preventive actionable insights can be realised. Moreover, with the availability of data overtime for training, gained expertise and active rules-set refinement with active feedback, the holistic model promotes interpretability, generalization, scalability and reliability in resource constrained environment. These are the **core advantages** of the proposed model over traditional RCA approaches that facilitates to make more informed decision making in the software testing paradigm.

CONCLUSION

This study aims to develop and evaluate an effective method to detect the potential root causes of the defects emerged during software testing through the leverage of metadata and ML-based RCA model. The main goal of this study was to create an automated, reliable, robust and generalized RCA model to overcome the limiting factors of pre-existing RCA methods. The scope of this study was made for businesses with limited or inadequate data and to enable them to draw insights and facilitate in making informed decisions.

Formulation of the research questions laid out the definition and scope of the defect-specific domain. It also aided in ensuring the proposed solution provided practicality and effectiveness and not based on accuracy alone.

In the experimental part of the study, the input data were collected from the metadata from JIRA tickets that included both structured and unstructured data. The author of this thesis applied rigorous preprocessing steps, once the data were extracted and combined, for maintaining the effective balance between the real-world defects' representation and defects management in a way that could be used by the chosen Machine Language (ML) models.

Eventually, the author introduced SentenceTransformer and TF-IDF vectorization for capturing the semantic relationships and its contextual nuances to promote the learning of the model from both structured and unstructured datatypes. The data preprocessing and feature engineering required the most significant effort as it significantly contributed for the models' inference and reliability.

Once the data were ready to be fed to the model. A total of 5 different ML algorithms from the wide spectrum of choices, through reasoning and justification led through literature reviews, were selected namely:

- *Random Forest (RF)*
- *Logistic Regression (LR)*
- *Support Vector Machines (SVM)*
- *Extreme Gradient Boosting (XGBoost)* and
- *Voting Classifier (Ensemble)*

To deal effectively with the contextual nuances and for output reliability, two modelling approaches were chosen by the author of this study, generic and tuned. The

generic models constituted of the standard ML algorithms focusing solely on structured metadata. The tuned models involved in hyperparameter optimization such as *RandomizedSearchCV / GridSearchCV* and *TF-IDF* incorporated by the author in the aim to delve deeper into the defect causes.

However, it was noticed that the *GridSearchCV* approach was more fine-grained when compared to *RandomizedSearchCV,* but due to the computational overhead, the author of this thesis chose to use the *RandomizedSearchCV* with further trimming to a total of 10 possible combinations instead of *GridSearchCV* that promoted to use all the possible combinations which demanded heavy resource utilization. The computationally intensive nature of *GridSearchCV* made it inefficient for the study intended organisational scope i.e., small businesses with limited data availability and resources.

The outcomes of the validation dataset illustrate ***XGBoost*** and ***Voting Classifier*** to produce the best results among the models that were considered. The Voting Classifier was the most effective in dealing with the multi-faceted class complexities, especially for rare classes.

It was noted by the author that with only classification of defects, it was inadequate for RCA to facilitate defect prevention and detection. Hence, a potential use case of the selected model, the last contribution of the study, was to incorporate a rule-based layer of RCA on top of the classifier's output.

The author aims to comprehend and provide valuable yet practical insights on the potential root causes through the use of an ***if-then*** statement that links the predicted labels to potential root causes which is an ideal choice for the organisations with limited data availability as it requires only the domain expertise, which can be improved progressively, providing a flexible approach to identify and validate the root causes interpretation.

The thesis **goal** has been successfully realised with the combination of highly accurate ML classifier and rule-based RCA engine, it provides to effectively and systematically translate the defect labels into cause hypothesis which and provide meaningful insights for corrective and preventive actions.

From section 3.5, it can also be understood that the study realised supplementary research questions. With further analysis, validity of the proposed model based on strengths and weaknesses can be identified. The ensemble model (Voting Classifier) showed excellent *accuracy* and *recall* scores during defect classification while adhering

to generalization and the nature of rule-based RCA (**if-then**). This made it readily manageable and interpretable even for businesses with limited data and resource availability.

However, it was observed that the domain-specific knowledge is required for the rule-based layer and the resource intensive incorporation of hyperparameter tuning for models such as *GridSearchCV*, may produce generic results if handling and management are not proper. Despite the effective handling capabilities of ensemble models, on occasions, it was observed that the very rare multi-faceted class complexities resulted in lower *recall*.

The **future research directions** based on the weakness are:

1. Investigate on the effective implementation of the techniques in real-time and in real-world scenarios.
2. Investigate on self-adapting or data-driven rules to leverage the use of NLP techniques for the dynamic update of the rules based upon the active feedback or through documented and resolved defects for proper cause and effect linkage automatically.
3. Investigate on effective hyperparameter settings based on organisational contexts for effective resource optimization to reduce the significant gap between the expected and the actual deliverables.

The author does not have permission to share the dataset used for the training, testing and validation of the models.

LIST OF REFERENCES

[1] K. Nishiura, T. Kasagi, and A. Monden, "A Cost-Effectiveness Metric for Association Rule Mining in Software Defect Prediction," in *Proceedings - 2023 Congress in Computer Science, Computer Engineering, and Applied Computing, CSCE 2023*, Institute of Electrical and Electronics Engineers Inc., 2023, pp. 2615–2620. doi: 10.1109/CSCE60160.2023.00418.

[2] A. K. Sharadhi, V. Gururaj, K. R. Umadi, M. Kumar, S. P. Shankar, and D. Varadam, "Comprehensive Survey of different Machine Learning Algorithms used for Software Defect Prediction," in *2022 International Conference on Decision Aid Sciences and Applications, DASA 2022*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 425–430. doi: 10.1109/DASA54658.2022.9764982.

[3] Shivam and K. Pal Sharma, "A Four-Stage dynamic approach for Software Defect Prediction," in *ICSCCC 2021 - International Conference on Secure Cyber Computing and Communications*, Institute of Electrical and Electronics Engineers Inc., May 2021, pp. 541–545. doi: 10.1109/ICSCCC51823.2021.9478165.

[4] Y. Gao, Y. Zhu, and Y. Zhao, "Dealing with imbalanced data for interpretable defect prediction," *Inf Softw Technol*, vol. 151, Nov. 2022, doi: 10.1016/j.infsof.2022.107016.

[5] S. Karthik, A. Sherry, and K. V Tej, "Software Defect Prevention through Defect Detection, Analysis, and Prediction: A Survey," 2016.

[6] Y. Qu, Z. Li, J. Zhao, and H. Li, "Unbalanced data processing for software defect prediction," in *2022 4th International Conference on Data-Driven Optimization of Complex Systems, DOCS 2022*, Institute of Electrical and Electronics Engineers Inc., 2022. doi: 10.1109/DOCS55193.2022.9967755.

[7] Y. Wang, S. Huang, Y. Xu, and H. Zhang, "Research on Software Defects Prediction and Peformance Analysis based on Machine Learning," in *2024 7th International Conference on Computer Information Science and Application Technology, CISAT 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 774–777. doi: 10.1109/CISAT62382.2024.10695196.

[8] N. Grattan, D. Alencar da Costa, and N. Stanger, “The need for more informative defect prediction: A systematic literature review,” Jul. 01, 2024, *Elsevier B.V.* doi: 10.1016/j.infsof.2024.107456.

[9] A. R. Ali, A. Ur Rehman, A. Nawaz, T. M. Ali, and M. Abbas, “An Ensemble Model for Software Defect Prediction,” in *2022 2nd International Conference on Digital Futures and Transformative Technologies, ICoDT2 2022*, Institute of Electrical and Electronics Engineers Inc., 2022. doi: 10.1109/ICoDT255437.2022.9787439.

[10] M. Solé, V. Muntés-Mulero, A. I. Rana, and G. Estrada, “Survey on Models and Techniques for Root-Cause Analysis,” Jan. 2017, [Online]. Available: http://arxiv.org/abs/1701.08546

[11] “Soft Skill Patterns for Software Developers: The ‘Learning from Unintended Failures’ Pattern - InfoQ.” Accessed: Nov. 04, 2024. [Online]. Available: https://www.infoq.com/articles/learning-unintended-failures-pattern/

[12] “Learning from Bugs and Testers: Testing Boeing 777 Full Flight Simulators - InfoQ.” Accessed: Nov. 04, 2024. [Online]. Available: https://www.infoq.com/articles/testing-boeing-flight-simulators-software/

[13] “PayPal Engineering Teams Implement Premortem Analysis - InfoQ.” Accessed: Nov. 04, 2024. [Online]. Available: https://www.infoq.com/news/2021/07/paypal-premortem-analysis/

[14] “Preview of AWS Cost Anomaly Detection Now Available - InfoQ.” Accessed: Nov. 04, 2024. [Online]. Available: https://www.infoq.com/news/2020/10/aws-cost-anomaly-detection/

[15] “5 Root Cause Analysis Examples That Shed Light on Complex Issues.” Accessed: Nov. 04, 2024. [Online]. Available: https://reliability.com/resources/articles/5-root-cause-analysis-examples/

[16] “A Beginners Guide To Root Cause Analysis (RCA) 2024 | Reliability.” Accessed: Nov. 04, 2024. [Online]. Available: https://reliability.com/resources/articles/beginners-guide-to-root-cause-analysis/

[17] “7 Root Causes for Software Defects and its Solutions | BrowserStack.” Accessed: Dec. 09, 2024. [Online]. Available: https://www.browserstack.com/guide/root-causes-for-software-defects-and-its-solutions

[18] M. F. Peerally, S. Carr, J. Waring, M. Dixon-Woods, and M. Farhad, "The problem with root cause analysis," 2016, doi: 10.1136/bmjqs-2016.

[19] "9 Powerful Best Root Cause Analysis Tools for Problem-Solving - RCA Tool." Accessed: Nov. 04, 2024. [Online]. Available: https://easyrca.com/blog/9-root-cause-analysis-tools/

[20] "Top Criticisms of the 5-Why Approach." Accessed: Nov. 04, 2024. [Online]. Available: https://blog.thinkreliability.com/top-criticisms-of-the-5-why-approach

[21] "9 Powerful Best Root Cause Analysis Tools for Problem-Solving - RCA Tool." Accessed: Nov. 04, 2024. [Online]. Available: https://easyrca.com/blog/9-root-cause-analysis-tools/

[22] "Fishbone Diagram, Types, Advantages, Drawbacks, and Six M's." Accessed: Nov. 04, 2024. [Online]. Available: https://bgmcgroup.com/exploring-the-fishbone-diagram-types-advantages-drawbacks-and-revealing-the-six-ms-of-change/

[23] "Fishbone Diagram Root Cause Analysis - Pros & Cons." Accessed: Nov. 04, 2024. [Online]. Available: https://taproot.com/fishbone-diagram-root-cause-analysis-pros-cons/

[24] "Ishikawa Diagram: Examples and Applications - Status.net." Accessed: Nov. 04, 2024. [Online]. Available: https://status.net/articles/ishikawa-diagram-examples-applications/

[25] "Fault tree analysis - Wikipedia." Accessed: Nov. 04, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Fault_tree_analysis

[26] "Reliability | PROACT Approach to RCA." Accessed: Nov. 04, 2024. [Online]. Available: https://reliability.com/proact-approach-to-rca/

[27] "Failure mode and effects analysis - Wikipedia." Accessed: Nov. 04, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Failure_mode_and_effects_analysis

[28] "Guide to Failure Mode and Effects Analysis (FMEA) | Reliability." Accessed: Nov. 04, 2024. [Online]. Available: https://reliability.com/resources/articles/guide-to-failure-mode-and-effects-analysis/

[29] "Top 10 Cons & Disadvantages of Pareto Charts." Accessed: Nov. 04, 2024. [Online]. Available: https://projectmanagers.net/top-10-cons-disadvantages-of-pareto-charts/

[30] "Pareto Analysis - Overview, Limitations, Pareto Diagram." Accessed: Nov. 04, 2024. [Online]. Available: https://corporatefinanceinstitute.com/resources/management/pareto-analysis/

[31] "Scatter Plot - Definition, Types, Analysis, Examples." Accessed: Nov. 04, 2024. [Online]. Available: https://www.cuemath.com/data/scatter-plot/

[32] "Scatter Diagrams (Plots), Analysis & Regression." Accessed: Nov. 04, 2024. [Online]. Available: https://sixsigmastudyguide.com/scatter-analysis-regression/

[33] "Scatterplots: Using, Examples, and Interpreting - Statistics By Jim." Accessed: Nov. 04, 2024. [Online]. Available: https://statisticsbyjim.com/graphs/scatterplots/

[34] "Affinity diagram - Wikipedia." Accessed: Nov. 04, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Affinity_diagram

[35] "Limitations of Six Sigma Steps and What to Do About Them | Lucidchart." Accessed: Nov. 04, 2024. [Online]. Available: https://www.lucidchart.com/blog/six-sigma-steps-and-their-limitations

[36] "The DMAIC Model | Problem Solving Training from EPM." Accessed: Nov. 04, 2024. [Online]. Available: https://expertprogrammanagement.com/2020/04/the-dmaic-model/

[37] S. Stradowski and L. Madeyski, "Machine learning in software defect prediction: A business-driven systematic mapping study," Mar. 01, 2023, *Elsevier B.V.* doi: 10.1016/j.infsof.2022.107128.

[38] X. Chen, D. Zhang, Y. Zhao, Z. Cui, and C. Ni, "Software defect number prediction: Unsupervised vs supervised methods," *Inf Softw Technol*, vol. 106, pp. 161–181, Feb. 2019, doi: 10.1016/j.infsof.2018.10.003.

[39] J. Kahles, J. Torronen, T. Huuhtanen, and A. Jung, "Automating root cause analysis via machine learning in agile software testing environments," in *Proceedings - 2019 IEEE 12th International Conference on Software Testing, Verification and Validation, ICST 2019*, Institute of Electrical and Electronics Engineers Inc., Apr. 2019, pp. 379–390. doi: 10.1109/ICST.2019.00047.

[40] Kazuya Tanaka, Akito Monden, and Zeynep Y¨ucel, *Prediction of Software Defects Using Automated Machine Learning*. IEEE Computer Society, 2019.

[41] M. Mustaqeem, S. Mustajab, and M. Alam, "Enhancing Software Defect Prediction Through Root Cause Analysis: A Hybrid Approach Integrating Permutation Importance with XGBoost (PERMBoost)," in *Proceedings of International Conference on Communication, Computer Sciences and Engineering, IC3SE 2024*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 61–67. doi: 10.1109/IC3SE62002.2024.10593406.

[42] F. Xu and Z. Sun, "Defect-Introducing Defect Prediction Testing," in *2024 IEEE 24th International Conference on Software Quality, Reliability, and Security Companion (QRS-C)*, IEEE, Jul. 2024, pp. 401–410. doi: 10.1109/QRS-C63300.2024.00057.

[43] Y. Zhou, "A Software Defect Prediction Approach based on Machine Learning," in *IEEE Advanced Information Technology, Electronic and Automation Control Conference (IAEAC)*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 630–634. doi: 10.1109/IAEAC59436.2024.10503608.

[44] X. Yu *et al.*, "Finding the best learning to rank algorithms for effort-aware defect prediction," May 01, 2023, *Elsevier B.V.* doi: 10.1016/j.infsof.2023.107165.

[45] Y. Shen, S. Hu, S. Cai, and M. Chen, "Software Defect Prediction based on Bayesian Optimization Random Forest," in *Proceedings - 2022 9th International Conference on Dependable Systems and Their Applications, DSA 2022*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 1012–1013. doi: 10.1109/DSA56465.2022.00149.

[46] G. Czibula, I. G. Chelaru, I. G. Czibula, and A. J. Molnar, "An unsupervised learning-based methodology for uncovering behavioural patterns for specific types of software defects," in *Procedia Computer Science*, Elsevier B.V., 2023, pp. 2644–2653. doi: 10.1016/j.procs.2023.10.256.

[47] P. Singh, N. Kim, R. Mendoza, A. Martinez, I. Cruz, and S. Patel, "Classification and Prediction in Data-Driven Analysis for Diverse Applications," Oct. 2024, doi: 10.13140/RG.2.2.16577.16481.

[48] S. K. Hamouri, R. A. Shatnawi, O. Alzoubi, A. Migdady, and M. B. Yassein, "Predicting Bug Severity Using Machine Learning and Ensemble Learning

Techniques," in *2023 14th International Conference on Information and Communication Systems, ICICS 2023*, Institute of Electrical and Electronics Engineers Inc., 2023. doi: 10.1109/ICICS60529.2023.10330494.

[49] P. Panda, D. Sahoo, and D. Sahoo, "Automating Fault Prediction in Software Testing Using Machine Learning Techniques: A Real-World Applications," in *2nd International Conference on Sustainable Computing and Smart Systems, ICSCSS 2024 - Proceedings*, Institute of Electrical and Electronics Engineers Inc., 2024, pp. 841–844. doi: 10.1109/ICSCSS60660.2024.10625524.

[50] Brittany Johnson, Yuriy Brun, and Alexandra Meliou, "Causal Testing: Understanding Defects' Root Causes," Jun. 27, 2020, *IEEE Computer Society*. doi: 10.1145/3377811.

[51] Y. Ran, S. Xiaomei, and X. Zhaowei, "Research and Application of Software Defect Prediction Model based on Data Mining," in *Proceedings of 2022 IEEE International Conference on Sensing, Diagnostics, Prognostics, and Control, SDPC 2022*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 367–371. doi: 10.1109/SDPC55702.2022.9915822.

[52] P. Tadapaneni, N. C. Nadella, M. Divyanjali, and Y. Sangeetha, "Software Defect Prediction based on Machine Learning and Deep Learning," in *5th International Conference on Inventive Computation Technologies, ICICT 2022 - Proceedings*, Institute of Electrical and Electronics Engineers Inc., 2022, pp. 116–122. doi: 10.1109/ICICT54344.2022.9850643.

[53] A. Detzner, R. Ruckschlos, and M. Eigner, "Root-Cause Analysis with Interactive Decision Trees," in *Proceedings of the International Conference on Information Visualisation*, Institute of Electrical and Electronics Engineers Inc., Sep. 2020, pp. 322–327. doi: 10.1109/IV51561.2020.00060.

[54] W. Albattah and M. Alzahrani, "Software Defect Prediction Based on Machine Learning and Deep Learning Techniques: An Empirical Approach," *AI*, vol. 5, no. 4, pp. 1743–1758, Sep. 2024, doi: 10.3390/ai5040086.

[55] Z. S. Alharthi, A. Alsaeedi, and W. M. S. Yafooz, "Software Defect Prediction Approaches: A Review," in *BioSMART 2021 - Proceedings: 4th International Conference on Bio-Engineering for Smart Technologies*, Institute of Electrical and Electronics Engineers Inc., 2021. doi: 10.1109/BioSMART54244.2021.9677869.

[56] "Voting Classifier - GeeksforGeeks." Accessed: Dec. 24, 2024. [Online]. Available: https://www.geeksforgeeks.org/voting-classifier/

[57] "20+ Most Widely Used Project Management Apps Compared." Accessed: Dec. 09, 2024. [Online]. Available: https://www.simplilearn.com/project-management-apps-article

[58] "20+ Agile Statistics: All About Agile Adoption | Runn." Accessed: Dec. 09, 2024. [Online]. Available: https://www.runn.io/blog/agile-statistics

[59] "9 Best Agile Project Management Tools | Atlassian." Accessed: Dec. 09, 2024. [Online]. Available: https://www.atlassian.com/agile/project-management/tools

[60] "Changing maxResults parameter for Jira Cloud REST API | Jira | Atlassian Documentation." Accessed: Dec. 09, 2024. [Online]. Available: https://confluence.atlassian.com/jirakb/changing-maxresults-parameter-for-jira-rest-api-779160706.html

[61] M. A. Syed, E. Arsevska, M. Roche, and M. Teissere, "A metadata approach to classify domain-specific documents for Event-based Surveillance Systems," in *2023 3rd International Conference on Communication, Computing and Digital Systems, C-CODE 2023*, Institute of Electrical and Electronics Engineers Inc., 2023. doi: 10.1109/C-CODE58145.2023.10139883.

[62] H. Khalid and E. Zimányi, "Repairing raw metadata for metadata management," *Inf Syst*, vol. 122, May 2024, doi: 10.1016/j.is.2024.102344.

[63] "re — Regular expression operations — Python 3.13.1 documentation." Accessed: Dec. 09, 2024. [Online]. Available: https://docs.python.org/3/library/re.html

[64] M. Neumann, D. King, I. Beltagy, and W. Ammar, "ScispaCy: Fast and Robust Models for Biomedical Natural Language Processing," Feb. 2019, doi: 10.18653/v1/W19-5034.

[65] "Natural Language Processing With spaCy in Python – Real Python." Accessed: Dec. 09, 2024. [Online]. Available: https://realpython.com/natural-language-processing-spacy-python/

[66] "NLP Library spaCy 3.0 Features Transformer-Based Models and Distributed Training." Accessed: Dec. 08, 2024. [Online]. Available: https://huggingface.co/facebook/bart-large-mnli

[67] "Python Libraries for NLP: NLTK, spaCy, and Transformers." Accessed: Dec. 08, 2024. [Online]. Available: https://www.analyticsinsight.net/python-2/python-libraries-for-nlp-nltk-spacy-and-transformers

[68] "Transformer (deep learning architecture) - Wikipedia." Accessed: Dec. 09, 2024. [Online]. Available: https://en.wikipedia.org/wiki/Transformer_(deep_learning_architecture)

[69] M. Lewis *et al.*, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," *Proceedings of the Annual Meeting of the Association for Computational Linguistics*, pp. 7871–7880, Oct. 2019, doi: 10.18653/v1/2020.acl-main.703.

[70] "IBM/zero-shot-classification-boost-with-self-training: code for the paper 'Zero-Shot Text Classification with Self-Training' for EMNLP 2022." Accessed: Dec. 08, 2024. [Online]. Available: https://github.com/IBM/zero-shot-classification-boost-with-self-training

[71] "facebook/bart-large-mnli · Hugging Face." Accessed: Dec. 08, 2024. [Online]. Available: https://huggingface.co/facebook/bart-large-mnli

[72] "VotingClassifier. A Voting Classifier is an ensemble… | by Ranjan kumar | Medium." Accessed: Dec. 24, 2024. [Online]. Available: https://medium.com/@ranjankumar_29097/votingclassifier-3f85ba8e4580

[73] Pirjatullah, D. Kartini, D. T. Nugrahadi, Muliadi, and A. Farmadi, "Hyperparameter Tuning using GridsearchCV on the Comparison of the Activation Function of the ELM Method to the Classification of Pneumonia in Toddlers," *Proceedings - 2021 4th International Conference on Computer and Informatics Engineering: IT-Based Digital Industrial Innovation for the Welfare of Society, IC2IE 2021*, pp. 390–395, 2021, doi: 10.1109/IC2IE53219.2021.9649207.

[74] R. Burduk *et al.*, "Progress on Pattern Classification, Image Processing and Communications," 2023.

[75] A. Tabassum and R. R. Patil, "A Survey on Text Pre-Processing & Feature Extraction Techniques in Natural Language Processing," *International Research Journal of Engineering and Technology*, 2020, [Online]. Available: www.irjet.net

[76] M. G. Kim and H. Kim, "Anomaly Detection in Imbalanced Encrypted Traffic with Few Packet Metadata-Based Feature Extraction," *CMES - Computer*

Modeling in Engineering and Sciences, vol. 141, no. 1, pp. 585–607, 2024, doi: 10.32604/cmes.2024.051221.

[77] "Semantic Textual Similarity — Sentence Transformers documentation." Accessed: Dec. 10, 2024. [Online]. Available: https://sbert.net/docs/sentence_transformer/usage/semantic_textual_similarity.html

APPENDIXES

Appendix 1

Code used for data preprocessing and cleaning

Draft 1:

```
from google.colab import drive

drive.mount('/content/drive')
import pandas as pd

data = pd.read_csv('/content/drive/MyDrive/Thesis/Data/structured_jira_issues.csv', header=None)

for index, column in enumerate(data.columns):
    print(f"Indexing: {index}, Names: {column}")

data.fillna('', inplace=True)

data.columns = data.iloc[0]
data = data[1:]

data.reset_index(drop=True, inplace=True)

drop_cols =
['StatusCategory_Name','Resolution_Name','Reporter_DisplayName','Reporter_TimeZone','Assignee_D
isplayName','Assignee_TimeZone']
df.drop(columns=drop_cols, axis=1, inplace=True)              #Dropping unrelevant rows

df['Created_Date'] = pd.to_datetime(df['Created_Date'], errors='coerce', utc=True)
df['Created_Date'] = df['Created_Date'].dt.date.astype(str)

import re
import spacy

nlp = spacy.load("en_core_web_sm")    #NER and Dependency Parsing

def text_clean(text):
  text = re.sub(r'http\S+|www\S+', 'URL', text)                              #Removing URLs, if present, in
"Summary"
  docs = nlp(text)                                                  #Tokenizing
  tokens = [token.lemma_ for token in doc if token.is_alpha and not token.is_stop]      #Stopwords for
removing stopwords as well as non-alphabetical words, if present

  cleaned = ' '.join(tokens)
  return cleaned              #Joining the tokens back into a string

def information(docs):        #Extracting some information using spacy
  cause_keywords = ['error', 'issue', 'bug', 'failure', 'problem', 'exception', 'failed', 'issues', 'not','missing',
'blocked']
```

```
 effect_keywords = ['crash', 'failure', 'down', 'timeout', 'slow', 'unavailable', 'broken', 'failed',
'issues','not','missing','blocked']

 cause = []
 effect = []

 for e in docs.ents:
   if e.label_ in ['ORG', 'PRODUCT', 'GPE', 'LOC']:              #These can be system components,
locations, etc.
          cause.append(e.text)
   elif e.label_ in ['EVENT', 'WORK_OF_ART', 'MONEY']:           #Effect-related events or outcomes
          effect.append(e.text)

 for token in docs:                                        #Contextual understanding
      if token.dep_ == 'ROOT':                                 #Identifying verbs that may indicate causes or
effects
          if token.lemma_ in cause_keywords:
             cause.append(token.text)
          elif token.lemma_ in effect_keywords:
             effect.append(token.text)

 for token in docs:
      if token.dep_ == 'dobj' and token.head.dep_ == 'ROOT':     #Checking relationship between verbs
and nouns, direct objects linked to the root verb
          if token.text.lower() in effect_keywords:
             effect.append(token.text)

 return ' '.join(set(cause)), ' '.join(set(effect))



file_path = '/content/drive/MyDrive/Thesis/Data/checkpoint1.csv'       #Some manual cleaning work was
done for "structured_jira_issues.csv" and saved as "checkpoint1.csv"
df = pd.read_csv(file_path)

df['summary'] = df['Summary'].apply(lambda x: text_clean(x))    #Preprocessing each observation

df[['cause', 'effect']] = df['summary'].apply(lambda x: information(nlp(x))).apply(pd.Series)    #Extraction
using spacy

summary = df[['Summary', 'cause', 'effect']]                #Validation


print(summary)
```

Draft 2:

```
file_path = '/content/drive/MyDrive/Thesis/Data/Labels.xlsx'      #2nd Draft
dataframe = pd.read_excel(file_path)

check = ['Description_Content', 'Labels', 'Comments']

#Removing rows where all the above columns are empty
masking = dataframe[check].apply(lambda row: row.str.strip().any() if row.apply(lambda x: isinstance(x,
str)).all() else row.notna().any(), axis=1)

dataframe = dataframe[masking]

labels = dataframe['Labels'].str.split(',').explode().str.strip()

all_unique_labels = labels.unique()
count_unique_labels = len(unique_labels)

print("Unique labels Count:", count_unique_labels)
print("All Unique Labels:", all_unique_labels)
```

Draft 3:

```
file_path = '/content/drive/MyDrive/Thesis/Data/Data.csv'        #3rd Draft
df = pd.read_csv(file_path)

df = df.drop(columns=['Linkage', 'Description_Content', 'Comments'], errors='ignore')        #Dropping
columns, as they provided no valid information

labels_manipulation = {
   'UI/ UX': 'UI/UX',
   'UI/UX': 'UI/UX',
   'UX': 'UI/UX',
   'UI': 'UI/UX',
   'Payment Configuration': 'Payment',
   'Checkout configuration': 'Checkout',
   'Checkout Shipping Address': 'Checkout',
   'Promotion': 'Promotions',
   'CHECKOUT': 'Checkout',
   'Checkout UI/UX': 'Checkout',
   'Maintainence': 'Maintenance',
   'Maintanence': 'Maintenance',
   'MAintainence': 'Maintenance',
   'CnR': 'BE Admin Level',
   'State Map': 'BE Admin Level',
        'HP': 'Home',
   'Homepage': 'Home',
```

```
    'BE Admin': 'BE Admin Level',
    'BE Admin Lvel': 'BE Admin Level',
    'stock source need set recently import product': 'ERP',
    'Sync Nicokick medium Live Staging June': 'Internal',
    'CARt': 'CART',
    'Structured Data': 'Code',
    'Layered Navigation': 'Layered Navigation',
    'Layered Navigration': 'Layered Navigation',
    'Layerd Navigation': 'Layered Navigation',
    'Filtered Navigation': 'Layered Navigation',
    'Patment': 'Payment',
    'Broken Link': 'Broken Links',
    'BE Level': 'BE Admin Level',
    'MY Account': 'My Account',
    'Shipment Method': 'Shipping Method',
    'Sign IN': 'Sign In',
    'Broken Links':'Brokern Links'
}

df_manipulation = df.reset_index(drop=True)
df_manipulation = df_manipulation.assign(Labels=df_manipulation['Labels'].str.split(','))     #Handle
combined labels after resetting index

df_manipulation = df_manipulation.explode('Labels')
df_manipulation['Labels'] = df_manipulation['Labels'].str.strip()                    #White-space removal
df_manipulation['Labels'] = df_manipulation['Labels'].replace(label_manipulation)       #Corrections for
handling standarized labels

df_manipulation = df_manipulation.drop_duplicates()

labels_sorted = sorted(df_manipulation ['Labels'].unique())
print(df_manipulation['Labels'])

label_count = df_manipulation['Labels'].nunique()
print(f"Unique labels: {label_count}")

original = df_manipulation.shape
df_manipulation = df_manipulation.drop_duplicates(subset=['Ticket_ID', 'Created_Date',
'Summary','Status_Name'], keep='first')

clean_shape = df_manipulation.shape
removedRows = original[0] - clean_shape[0]

print(f"Before: {original[0]}")
print(f"After: {clean_shape[0]}")
print(f"Removed: {removedRows}")

uniqueLabel_count = df_manipulation['Labels'].nunique()
```

```
print(f"\nlabelCount: {uniqueLabel_count}")

Labels_sorted = sorted(df_manipulation['Labels'].unique())

print("\nSorted:")
print(Labels_sorted)

empty = df_manipulation['Labels'].isna().sum() + (df_manipulation['Labels'] == '').sum()

filled = df_manipulation['Labels'].notna().sum() - (df_manipulation['Labels'] == '').sum()

print(f"Empty: {empty}")
print(f"Filled: {filled}")

df_manipulation['Labels'] = df_manipulation['Labels'].astype(str).str.strip()
df = df_manipulation.groupby(
    ['Ticket_ID', 'Ticket_Key', 'Summary', 'Created_Date', 'Status_Name', 'Priority_Name'],as_index =
False
)['Labels'].agg(lambda x: ', '.join(sorted(set(x))))

df = df.reset_index(drop=True)


status = {
    'PM Action (migrated)': 'PM Action',
    'Release to Live (migrated)': 'Release to Live',
    'Client Acceptance Of The Done Work': 'Done',
    'Closed': 'Done',
    'On Hold': 'Client Action',
    'Ready To Do': 'New',
    'QA on Live (migrated)': 'QA on LIVE',
    'Release to Staging': 'QA on Staging',
    'QA on DEV': 'QA on Staging',
    'Scoping': 'In Progress',
    'Release to DEV': 'QA on Staging',
    'Code review': 'In Progress',
    'Release to Live': 'QA on LIVE'
}

df['Status_Name'] = df['Status_Name'].replace(status)
print(df['Status_Name'].unique())

from spacy.training import Example

nlp = spacy.load("en_core_web_sm")

def classification(text):                          #Providing manual classification for known labels
```

```
    frontend = ['UI', 'UX', 'Frontend', 'UI/UX', 'PDP', 'PLP', 'SEO', 'Home', 'Navigation', 'Payment', 'Search',
'Layered Navigation', 'Pop Up', 'Promotions']
    backend = ['API', 'ERP', 'Sync', 'Backend', 'Server', 'Code', 'Database', 'Security', 'Performance']

    text = text.lower()

    if any(keyword.lower() in text for keyword in frontend):
        return 'Frontend'
    elif any(keyword.lower() in text for keyword in backend):
        return 'Backend'

    return 'Unknown'                            #Label as "Unknown" if no keyword found

df['Elements'] = df['Labels'].apply(classification)
df['Elements'] = df.apply(lambda row: classification(row['Summary']) if row['Elements'] == 'Unknown' else
row['Elements'],axis=1

print(df[['Labels', 'Elements']].head())

# To classify the "Unknown" with using Hugging Face zero-shot classification model
from transformers import pipeline

classification = pipeline("zero-shot-classification", model="facebook/bart-large-mnli")        #Pre-trained
Classification model

classify_labels = ['Frontend', 'Backend']                                                    #Defining labels

def labels_element(text):                                                          #Function to classify "Frontend"
and "Backend"
    result = classification(text, classify_labels)
    return result['labels'][0]                                                  # Return the label with the highest
score

df['Elements'] = df['Elements'].replace('Unknown', '')

df['Elements'] = df.apply(lambda row: labels_element (row['Labels']) if row['Elements'] == '' else
row['Elements'], axis=1)

#Generate Task Description based on Status_Name column

def user_story(row):
    status = row['Status_Name']
    labels = row['Labels']
    summary = row['Summary']
    Elements = row['Elements']

    Task_Description = ""
```

```
    if status == 'Done': Task_Description = "The issue has been resolved, and the expected outcome has been achieved."
    elif status == 'Log time': Task_Description = "Time logs have been recorded, but further action may be required."
    elif status == 'New': Task_Description = "This is a new issue that needs to be investigated."
    elif status == 'Client Action': Task_Description = "The user stories were not clear or optimized, which caused delays or misunderstandings."
    elif status == 'PM Action': Task_Description = "The lack of clear understanding of the user stories led to the issue. The project manager needs to ensure clarity."
    elif status == 'Ready for development': Task_Description = "The issue was ready for development, but there were some ambiguities or anomalies in the requirements."
    elif status == 'Failed QA': Task_Description = "The actual and expected outcome did not align. The developer must address the discrepancies."
    elif status == 'QA on LIVE': Task_Description = "The issue is in the QA phase on live, indicating a need for immediate attention to resolve any discrepancies."
    elif status == 'In Progress': Task_Description = "The issue is being worked on, but further investigation is required."
    elif status == 'QA on UAT': Task_Description = "The issue is under testing in UAT, which indicates progress towards finalization."
    elif status == 'QA on Staging': Task_Description = "The issue is in QA on the staging environment, requiring final checks before release."
    elif status == 'Release to Live': Task_Description = "The issue has been resolved and is being released to live, indicating completion."
    else:
        Task_Description = "The issue is still under investigation or action."

    return Task_Description

df['Task_Description'] = df.apply(user_story, axis=1)

csv_filename = 'Cleaned.csv'
df.to_csv(csv_filename, index=False)

excel_filename = 'Cleaned.xlsx'
df.to_excel(excel_filename, index=False)
```

Draft 4:

```
file_path = '/content/Cleaned.csv'        #4th Draft
df = pd.read_csv(file_path)

unique_labels = df['Priority_Name'].unique()
print(unique_labels)


#Manual verification and validation of the dataset
```

Draft 5:

```
import pandas as pd

file_path = '/content/drive/MyDrive/Thesis/Data/cleanData.csv'        #5th Draft
ignore = ['Ticket_ID', 'Ticket_Key', 'Summary', 'Created_Date', 'Task description']    #Ignoring these
columns at the moment

unique_labels = {col: df[col].nunique() for col in df.columns if col not in ignore }

print("Total Count after ignoring irrelevant cols:")
for col, count in unique_labels.items():
    print(f"{col}: {count}")

totalPriority_count = df['Priority_Name'].value_counts()

print("Total Unique Levels:", totalPriority_count.shape[0])
print(totalPriority_count)

status_count = df['Status_Name'].value_counts()

print("Total Unique Levels:", status_count.shape[0])
print(status_count)

labels_count = df['Labels'].value_counts()

print("Unique Levels:", labels_count.shape[0])
print(labels_count)

#For Class Imbalance

from imblearn.over_sampling import SMOTE
from sklearn.preprocessing import LabelEncoder
from collections import Counter

label_encoder_priority = LabelEncoder()
df['Priority_Name_1'] = label_encoder_priority.fit_transform(df['Priority_Name'])

X_priority = df.drop(columns=['Priority_Name', 'Priority_Name_1', 'Status_Name', 'Labels'])

y_priority = df['Priority_Name_1']

X_priority_encoded = pd.get_dummies(X_priority, drop_first=True)

class_size_priority = min(Counter(y_priority).values())

smote_priority = SMOTE(random_state=42, k_neighbors=min(class_size_priority - 1, 5))
X_priority_smote, y_priority_smote = smote_priority.fit_resample(X_priority_encoded, y_priority)
print(Counter(y_priority_smote))
```

```
labels_to_group = ['Log time', 'New', 'In Progress', 'Release to Live']
df['Status_Name_1'] = df['Status_Name'].apply(lambda x: x if x not in labels_to_group else 'Other')

label_encoder_status = LabelEncoder()
df['Status_Name_encoded'] = label_encoder_status.fit_transform(df['Status_Name_1'])

X_status = df.drop(columns=['Status_Name', 'Status_Name_1', 'Status_Name_encoded',
'Priority_Name', 'Labels'])
y_status = df['Status_Name_encoded']

X_status_encoded = pd.get_dummies(X_status, drop_first=True)

class_size_status = min(Counter(y_status).values())
smote_status = SMOTE(random_state=42, k_neighbors=min(class_size_status - 1, 5))
X_status_smote, y_status_smote = smote_status.fit_resample(X_status_encoded, y_status)

print(Counter(y_status_smote))

rare_label_threshold = 50
label_counts = df['Labels'].value_counts()
df['Labels_1'] = df['Labels'].apply(lambda x: x if label_counts[x] >= rare_label_threshold else 'Other')

label_encoder_labels = LabelEncoder()
df['Labels_encoded'] = label_encoder_labels.fit_transform(df['Labels_1'])

X_labels = df.drop(columns=['Labels', 'Labels_1', 'Labels_encoded', 'Priority_Name', 'Status_Name'])
y_labels = df['Labels_encoded']

X_labels_encoded = pd.get_dummies(X_labels, drop_first=True)

class_size_labels = min(Counter(y_labels).values())
smote_labels = SMOTE(random_state=42, k_neighbors=min(class_size_labels - 1, 5))
X_labels_smote, y_labels_smote = smote_labels.fit_resample(X_labels_encoded, y_labels)

print(Counter(y_labels_smote))

class_size_labels = min(Counter(y_labels).values())
smote_labels = SMOTE(random_state=42, k_neighbors=min(class_size_labels - 1, 5))
X_labels_smote, y_labels_smote = smote_labels.fit_resample(X_labels_encoded, y_labels)

print(Counter(y_labels_smote))

df.to_csv('cleanData.csv', index=False)
df.to_excel('cleanData.xlsx', index=False)
```

Priority_Name class distribution after SMOTE:
Counter({2: 1865, 0: 1865, 1: 1865, 3: 1865, 4: 1865})

Status_Name class distribution after SMOTE:
Counter({1: 4289, 3: 4289, 0: 4289, 7: 4289, 5: 4289, 8: 4289, 2: 4289, 6: 4289, 4: 4289})

Labels class distribution after SMOTE:

Counter({23: 749, 4: 749, 14: 749, 5: 749, 15: 749, 1: 749, 13: 749, 10: 749, 12: 749, 6: 749, 2: 749, 19: 749, 22: 749, 21: 749, 7: 749, 0: 749, 9: 749, 11: 749, 24: 749, 18: 749, 8: 749, 20: 749, 3: 749, 17: 749, 16: 749}

```
Target Mappings:

Target: Priority_Name
0 -> Blocker 24/7
1 -> Critical
2 -> High
3 -> Low
4 -> Medium

Target: Status_Name
0 -> Client Action
1 -> Done
2 -> Failed QA
3 -> In Progress
4 -> Log time
5 -> New
6 -> PM Action
7 -> QA on LIVE
8 -> QA on Staging
9 -> QA on UAT
10 -> Ready for development
11 -> Release to Live

Features and targets prepared.
```

```
Encoded Label: 0 -> Original Label: API
Encoded Label: 1 -> Original Label: BE Admin Level
Encoded Label: 2 -> Original Label: Brokern Links
Encoded Label: 3 -> Original Label: CART
Encoded Label: 4 -> Original Label: CMS
Encoded Label: 5 -> Original Label: CRON
Encoded Label: 6 -> Original Label: Checkout
Encoded Label: 7 -> Original Label: Code
Encoded Label: 8 -> Original Label: Conversion
Encoded Label: 9 -> Original Label: Create Account
Encoded Label: 10 -> Original Label: Cross Browser
Encoded Label: 11 -> Original Label: Cross Device
Encoded Label: 12 -> Original Label: Downtime
Encoded Label: 13 -> Original Label: ERP
Encoded Label: 14 -> Original Label: Footer
Encoded Label: 15 -> Original Label: Home
Encoded Label: 16 -> Original Label: Internal
Encoded Label: 17 -> Original Label: Layered Navigation
Encoded Label: 18 -> Original Label: Maintenance
Encoded Label: 19 -> Original Label: My Account
Encoded Label: 20 -> Original Label: Navigation
Encoded Label: 21 -> Original Label: Newsletter
Encoded Label: 22 -> Original Label: PDP
Encoded Label: 23 -> Original Label: PLP
Encoded Label: 24 -> Original Label: Payment
Encoded Label: 25 -> Original Label: Performance
Encoded Label: 26 -> Original Label: Pop Up
Encoded Label: 27 -> Original Label: Promotions
Encoded Label: 28 -> Original Label: SEO
Encoded Label: 29 -> Original Label: Search
Encoded Label: 30 -> Original Label: Security
Encoded Label: 31 -> Original Label: Shipping Address
Encoded Label: 32 -> Original Label: Shipping Method
Encoded Label: 33 -> Original Label: Sign In
Encoded Label: 34 -> Original Label: Sitemap
Encoded Label: 35 -> Original Label: Sync
Encoded Label: 36 -> Original Label: UI/UX
Encoded Label: 37 -> Original Label: URL Migration
Encoded Label: 38 -> Original Label: Upgrade
Encoded Label: 39 -> Original Label: Usability
```

Appendix 2

Code used without Hyperparameter tuning

SentenceTransformer for generating embeddings to find the semantic relationship

```
import numpy as np
import json
import warnings
warnings.filterwarnings("ignore")

from sklearn.exceptions import UndefinedMetricWarning
from sklearn.preprocessing import LabelEncoder
from sentence_transformers import SentenceTransformer


def embeddings(input_csv, output_csv):
    df = pd.read_csv(input_csv)
    model = SentenceTransformer('all-MiniLM-L6-v2')

      summary_embeddings = model.encode(df['Summary'].astype(str).tolist(), show_progress_bar=True)
    df['Summary_embeddings'] = list(summary_embeddings)

    task_description_embeddings = model.encode(df['Task_Description'].astype(str).tolist(),
show_progress_bar=True)
    df['Task_Description_embeddings'] = list(task_description_embeddings)

    df.to_csv(output_csv, index=False)
    print(f"Saved: '{output_csv}'.")

embeddings("cleanData.csv", "cleanData_with_embeddings.csv")

def expand_embeddings(input_csv, output_csv):
    data = pd.read_csv(input_csv)

    def parse(value):
        try:
            return np.array([float(i) for i in value.strip("[]").replace("\n", " ").replace(",", " ").strip().split()])
        except Exception:
            return np.zeros(384)

    for column, prefix in zip(['Summary_embeddings', 'Task_Description_embeddings'], ['Summary',
'Task_Description']):
        data[column] = data[column].apply(parse)
        expanded = pd.DataFrame(data[column].tolist(), columns=[f"{prefix}_emb_{i}" for i in
range(data[column][0].size)])
        data = pd.concat([data, expanded], axis=1)
        data.drop(columns=[column], inplace=True)
```

```
    data.to_csv(output_csv, index=False)
    print(f"Embeddings expanded and saved as '{output_csv}'.")

expand_embeddings("cleanData_with_embeddings.csv",
"cleanData_with_expanded_embeddings.csv")

def features_and_targets(input_csv):
    data = pd.read_csv(input_csv)

    features = [col for col in data.columns if col.startswith(('Elements', 'Labels', 'Priority_Name_encoded',
'Status_Name_grouped', 'Labels_combined', 'Summary_embeddings_',
'Task_Description_embeddings_'))]

    targets = ['Priority_Name', 'Status_Name', 'Labels', 'Summary','Task_Description']

    categories = ['Elements', 'Labels', 'Priority_Name_encoded', 'Status_Name_grouped',
'Labels_combined']
    label_encoders = {}
    for col in categories:
        if col in data.columns:
            label_encoders[col] = LabelEncoder()
            data[col] = label_encoders[col].fit_transform(data[col])

    for target in targets:
        if data[target].dtype == 'object':
            label_encoders[target] = LabelEncoder()
            data[target] = label_encoders[target].fit_transform(data[target])
    return data, features, targets

data, features, targets = features_and_targets("cleanData_with_expanded_embeddings.csv")
```

Splitting the dataset for training and testing

```
from sklearn.model_selection import train_test_split
import numpy as np

def split(data, features, target):

    X = data[features] y = data[target]
    try:
        X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
        X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.33, random_state=42,
stratify=y_temp)
    except ValueError:
        print("Using random splitting, rare class detected")
        X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
        X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.33, random_state=42)
    return X_train, X_val, X_test, y_train, y_val, y_test
```

Model Training

```
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from xgboost import XGBClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC

def prepare():
    rf_model = RandomForestClassifier(random_state=42, n_estimators=100, class_weight='balanced')
    xgb_model = XGBClassifier(random_state=42, eval_metric='mlogloss')
    lr_model = LogisticRegression(max_iter=5000, random_state=42)
    svm_model = SVC(probability=True, random_state=42)
    voting_model = VotingClassifier(estimators=[
        ('rf', rf_model),
        ('xgb', xgb_model),
        ('lr', lr_model),
        ('svm', svm_model)], voting='soft')
    return rf_model, xgb_model, lr_model, svm_model, voting_model

def train(models, X_train, y_train):
    for model in models:
        model.fit(X_train, y_train)
def evaluate(models, X, y, phase):
    results = {}
    model_names = ["Random Forest", "XGBoost", "Logistic Regression", "SVM", "Voting Classifier"]
    for model, name in zip(models, model_names):
        results[name] = evaluate_model(f"{name} ({phase})", y, model.predict(X))
    return results

def train_and_evaluate(data, features, targets):

    for target in targets:
        if target in ["Summary", "Task_Description"]:
            print(f"\nSkipping target: {target}")
            continue
        X = data[features]
        y = data[target]

        try:
            X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42,
stratify=y)
            X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.33,
random_state=42, stratify=y_temp)
        except ValueError:
            print("Rare classes detected, using random splitting instead.")
            X_train, X_temp, y_train, y_temp = train_test_split(X, y, test_size=0.3, random_state=42)
            X_val, X_test, y_val, y_test = train_test_split(X_temp, y_temp, test_size=0.33,
random_state=42)
```

```
    X_train.fillna(X_train.mean(), inplace=True)
    X_val.fillna(X_val.mean(), inplace=True)
    X_test.fillna(X_test.mean(), inplace=True)

    rf_model = RandomForestClassifier(random_state=42, n_estimators=100,
class_weight='balanced')
    xgb_model = XGBClassifier(random_state=42, eval_metric='mlogloss')
    lr_model = LogisticRegression(max_iter=5000, random_state=42)
    svm_model = SVC(probability=True, random_state=42)
    voting_model = VotingClassifier(estimators=[('rf', rf_model), ('xgb', xgb_model), ('lr', lr_model),
('svm', svm_model)], voting='soft')

    rf_model.fit(X_train, y_train)
    xgb_model.fit(X_train, y_train)
    lr_model.fit(X_train, y_train)
    svm_model.fit(X_train, y_train)
    voting_model.fit(X_train, y_train)

    evaluate ("Random Forest (Validation)", y_val, rf_model.predict(X_val))
    evaluate ("XGBoost (Validation)", y_val, xgb_model.predict(X_val))
    evaluate ("Logistic Regression (Validation)", y_val, lr_model.predict(X_val))
    evaluate ("SVM (Validation)", y_val, svm_model.predict(X_val))
    evaluate ("Voting Classifier (Validation)", y_val, voting_model.predict(X_val))

    print("\nTest Scores:")
    evaluate ("Random Forest (Test)", y_test, rf_model.predict(X_test))
    evaluate ("XGBoost (Test)", y_test, xgb_model.predict(X_test))
    evaluate ("Logistic Regression (Test)", y_test, lr_model.predict(X_test))
    evaluate ("SVM (Test)", y_test, svm_model.predict(X_test))
    evaluate ("Voting Classifier (Test)", y_test, voting_model.predict(X_test))

train_and_evaluate(data, features, targets)
```

Evaluation of model performance on Validation dataset

```
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score,
classification_report, confusion_matrix

def evaluate (model_name, y_true, y_pred):
  print(f"{model_name}")
  accuracy = accuracy_score(y_true, y_pred)
  precision = precision_score(y_true, y_pred, average='weighted', zero_division=0)
  recall = recall_score(y_true, y_pred, average='weighted', zero_division=0)
  f1 = f1_score(y_true, y_pred, average='weighted', zero_division=0)

  print(f"{accuracy:.4f}")
  print(f"{precision:.4f}")
```

```
print(f"{recall:.4f}")
print(f"{f1:.4f}")
print("\nClassification Report:")
print(classification_report(y_true, y_pred, zero_division=0))
print("\nConfusion Matrix:")
print(confusion_matrix(y_true, y_pred))
return {
    "Accuracy": accuracy,
    "Precision": precision,
    "Recall": recall,
    "F1 Score": f1
}
```

Appendix 3

Code used with Hyperparameter tuning

```
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier, VotingClassifier
from xgboost import XGBClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.svm import SVC
from sklearn.metrics import classification_report, roc_auc_score, f1_score
from sklearn.preprocessing import LabelEncoder
from sklearn.model_selection import train_test_split, RandomizedSearchCV
from imblearn.under_sampling import RandomUnderSampler
from imblearn.over_sampling import SMOTE
from sklearn.pipeline import Pipeline
from sklearn.feature_extraction.text import TfidfVectorizer

data = pd.read_csv('/kaggle/input/dataset/cleanData_with_embeddings.csv')

#Appending contextual features

data['Combined_Text'] = data['Summary'] + " " + data['Task_Description']

label_encoder = LabelEncoder()
y_encoded = label_encoder.fit_transform(data['Labels'])

vectorizer = TfidfVectorizer(max_features=1000)
X = vectorizer.fit_transform(data['Combined_Text']).toarray()

X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y_encoded, test_size=0.2, stratify=y_encoded, random_state=42
)
X_train, X_val, y_train, y_val = train_test_split(
    X_train_val, y_train_val, test_size=0.25, stratify=y_train_val, random_state=42
)
over_sampler = SMOTE(random_state=42, k_neighbors=2)
under_sampler = RandomUnderSampler(random_state=42)
X_resampled, y_resampled = over_sampler.fit_resample(X_train, y_train)
X_resampled, y_resampled = under_sampler.fit_resample(X_resampled, y_resampled)

models = {
    'RandomForest': RandomForestClassifier(class_weight="balanced", random_state=42),
    'LogisticRegression': LogisticRegression(class_weight="balanced", max_iter=500, random_state=42),
    'SVM': SVC(probability=True, class_weight="balanced", random_state=42),
    'XGBoost': XGBClassifier(random_state=42, eval_metric='mlogloss'),
    'VotingClassifier': VotingClassifier(
        estimators=[
            ('rf', RandomForestClassifier(class_weight="balanced", random_state=42)),
```

```
        ('lr', LogisticRegression(class_weight="balanced", max_iter=500, random_state=42)),
        ('svm', SVC(probability=True, class_weight="balanced", random_state=42)),
        ('xgb', XGBClassifier(random_state=42, eval_metric='mlogloss'))
    ],
    voting='soft'
  )
}

#Hyperparameter tuning

param_grids = {
  'RandomForest': {
    'n_estimators': [100, 200],
    'max_depth': [10, 20, None],
    'min_samples_split': [2, 5, 10]
  },
  'LogisticRegression': {
    'C': [0.1, 1, 10]
  },
  'SVM': {
    'C': [0.1, 1, 10],
    'kernel': ['linear', 'rbf']
  },
  'XGBoost': {
    'n_estimators': [100, 200],
    'learning_rate': [0.01, 0.1, 0.3],
    'max_depth': [3, 5, 7]
  },
  'VotingClassifier': {}
}

results = []

for model_name, model in models.items():
  print(f"Training {model_name}...")
  pipeline = Pipeline(steps=[
    ('classifier', model)
  ])

  param_grid = param_grids.get(model_name, {})

  random_search = RandomizedSearchCV(
    pipeline,
    param_distributions={'classifier__' + key: value for key, value in param_grid.items()},
    scoring='f1_weighted',
    cv=3,
    verbose=2,
    n_jobs=-1,
    n_iter=10  #10 random combinations
```

```
    )

    random_search.fit(X_resampled, y_resampled)

    best_model = random_search.best_estimator_

    y_val_pred = best_model.predict(X_val)
    y_val_proba = best_model.predict_proba(X_val) if hasattr(best_model, "predict_proba") else None

    f1_val = f1_score(y_val, y_val_pred, average='weighted')
    roc_auc_val = roc_auc_score(y_val, y_val_proba, multi_class="ovr") if y_val_proba is not None else
"N/A"

    print(f"{model_name}")
    print(classification_report(y_val, y_val_pred))
    print(f"F1-Score (Validation): {f1_val:.4f}")
    print(f"ROC-AUC (Validation): {roc_auc_val}\n")

    results.append({
        "Model": model_name,
        "Best Params": random_search.best_params_,
        "F1-Score (Validation)": f1_val,
        "ROC-AUC (Validation)": roc_auc_val
    })

results_df = pd.DataFrame(results)
```